

USER MANUAL

Maprisma

& AI Agents Map

Your AI agents, local models, MCP servers and skills on one map

Maprisma 0.12.0 · AI Agents Map 0.26.1

October 2026

About this manual

This manual describes one application that comes in two editions:

Edition	Maprisma	AI Agents Map
What it is	A stand-alone GTK 4 / libadwaita app for any Linux desktop	A plugin for the Omarchy desktop, opened from a button in the bar
Install from	AUR, AppImage or source	The Omarchy Plugin Marketplace
Look	Follows the system's light/dark mode and accent colour	Omarchy theme colours, or GNOME's Adwaita look
Settings folder	`~/.config/maprisma/`	`~/.config/omarchy/ai-agents-map/`

Everything else — Setup, the map, connecting by drawing, local LLMs, models, MCP servers and skills — is the same in both. Where something applies to only one edition, it is marked MAPRISMA or OMARCHY.

Conventions

Add	A button, tab or menu item in the window.
Settings › **Look**	A path through the interface: open Settings, then **Look**.
`~/.agents/skills`	A file, folder or command. `~` is your home folder.
`Ctrl`+`,`	A keyboard shortcut.

> **NOTE** Notes add useful background.

> **TIP** Tips show a quicker way or a good habit.

> **CAUTION** Cautions point out something that is hard to undo.

Contents

1 Introduction 1
1.1 What it does 1
1.2 Key ideas 2
1.3 Nothing changes without a preview 2
2 Installation 3
2.1 Maprisma 3
2.2 AI Agents Map on Omarchy 3
2.3 First start 3
2.4 Removing the app 4
3 The window at a glance 5
3.1 Setup 6
3.2 Choosing what the map shows 8
3.3 Working with the map 9
3.4 Open problems 10
3.5 Open problems in the Flutter preview 10
3.6 Updates and suggestions 11
3.7 Add-on suggestions in the Flutter preview 11
4 Agents 12
4.1 Agent details 12
4.2 Starting and stopping agents 13
4.3 Network traffic 14
4.4 Subagents 14
4.5 Agents working together in the Flutter preview 15
4.6 Costs in the Flutter preview 15
4.7 Details in the Flutter preview 15
4.8 Terminal tabs, the usage box and the starter model in the Flutter preview 16
4.9 Configure… 16
4.10 Signing in 16
4.11 API keys in the Flutter preview 17
4.12 Updating agents 17
4.13 Updating a running agent in the Flutter preview 18
4.14 Agents the app can install 18
4.15 New agents 18
5 Local LLMs 19
5.1 Installing a local LLM 19
5.2 Installing in the Flutter preview 19
5.3 Starting and stopping with agents 20
5.4 A local LLM's details 20
5.5 Performance 20
5.6 Performance in the Flutter preview 21
5.7 GPU memory and Free GPU memory 21
5.8 Models at once 21
5.9 Models at once in the Flutter preview 22
5.10 Used by other apps 22
6 Models 24
6.1 Get from Hugging Face 24
6.2 Interrupted downloads in the Flutter preview 25
6.3 Ask an agent which model fits 25
6.4 Installed 25
6.5 Context size 25
6.6 On this PC 26
6.7 Moving and deleting models 26
6.8 One models folder 26
7 Connecting by drawing 27
7.1 Agent ↔ local LLM 27
7.2 Agent ↔ MCP server 27
7.3 Agent ↔ skill folder 27
7.4 Found on this PC 27
7.5 Local connections in the Flutter preview 28
7.6 Found on this PC in the Flutter preview 28
7.7 When a connection fails 29
8 MCP servers 30
8.1 Where servers come from 30
8.2 Setting a server up 30
8.3 MCP servers in the Flutter preview 31
8.4 Managing servers 31
9 Skills 32
9.1 Finding skills 32
9.2 Skills from the web in the Flutter preview 32
9.3 Skill folders 32
10 Settings and Help 33
10.1 Flutter themes and map lights 33
10.2 The popup under the bar on Omarchy 33
10.3 Usage figures and corners from Omarchy in the Flutter preview 33
10.4 Settings 33
10.5 Red limits in the Flutter preview 35
10.6 Recent changes and Undo after a restart 35
10.7 Help 36
10.8 Help in the Flutter preview 36
10.9 Disk space 36
11 Smart Check 38
11.1 PC setup summary 39

11.2 Updates found 39
11.3 While an agent works 39
11.4 The local helper 40
11.5 AI checks and the local helper in the Flutter preview 40
11.6 Protocol 40
11.7 Token statistics 41
12 Group cards and add-ons on the map 42
12.1 Expand a category 42
12.2 Manage add-ons 42
12.3 First use and existing flows 42
13 Editing the Flutter AI Map 43
14 Task cards on the map 44
15 System Map 45
15.1 First use and separate mounts 45
15.2 Saved map and refresh 45
15.3 Reconnect, move and undo 45
15.4 Analysis and AI help 45
16 Healthy & Security check 46
16.1 Reading the cards 46
16.2 Changing a setting 47
16.3 Security in the Flutter preview 47
16.4 Health in the Flutter preview 47
16.5 What is checked for each agent 48
16.6 Managing the risk 48
16.7 The words explained 49
17 Add-ons 51
17.1 Setting up Maprisma 51
17.2 The Add-ons page 52
17.3 Installing, switching on and off, updating, removing 53
17.4 Privacy levels 53
17.5 Kinds of add-ons 53
17.6 A new agent on your computer 54
17.7 Making an add-on 54
18 Privacy and safety 55
19 Troubleshooting 56
A Reference 57
A.1 Keyboard shortcuts 57
A.2 Keyboard use in the Flutter preview 57
A.3 Files and folders 57
A.4 Glossary 58

CHAPTER 1

Introduction

Maprisma shows the AI setup on your computer as cards and as a map: which coding agents are installed, which language models they use — cloud services and local runtimes — the models those serve, and which MCP servers and skills each agent has. And it lets you change that setup by drawing lines.

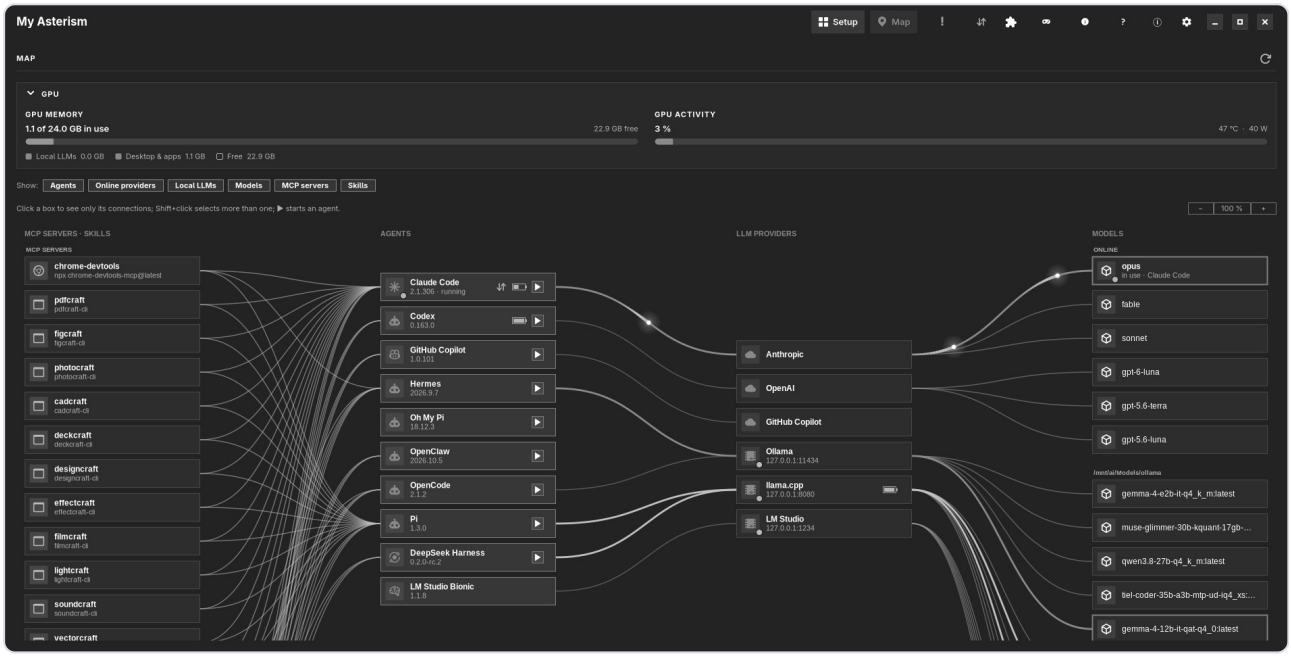


Figure 1 · The main window: the map across the whole width, the GPU card above it, the ! for open problems in the header.

1.1 What it does

- **Coding agents** — Claude Code, Codex, GitHub Copilot, Cursor CLI, Gemini, OpenCode, Pi, Hermes and more: their version, plan, limits and usage, and what each one still needs, with the command that fixes it.
- **Local LLMs** — llama.cpp, Ollama, LM Studio, vLLM, Jan and KoboldCpp: install them, download models, set the context size that fits your graphics card, and start them together with the agents that use them.
- **MCP servers and skills** — found on your computer or on the web, with recommended servers and a search of the official MCP registry.
- **Open problems** — a ! in the header when something needs fixing, each problem with the way to where it is fixed, and what your setup is good for.

1.2 Key ideas

Agent	A program you talk to that writes code or does tasks, such as Claude Code.
LLM provider	Where an agent's model runs: a cloud service (Anthropic, OpenAI, …) or a local LLM runtime on this computer (llama.cpp, Ollama, …).
Model	The language model itself. Local models are files, usually in GGUF format.
MCP server	A small program or web service that gives agents extra tools — reading files, searching the web, controlling a browser. MCP is the Model Context Protocol.
Skill	A folder with instructions (a `SKILL.md`) and sometimes scripts that teach an agent how to do a particular job.

1.3 Nothing changes without a preview

Every change to an agent's settings is shown first, line by line, before anything is written. The old file is kept as a backup, and the bar that appears after the change has **Undo**. The app itself never runs with administrator rights; only installing an agent that comes as a system package asks for your password, in a terminal you watch.

CHAPTER 2

Installation

2.1 Maprisma MAPRISMA

Maprisma also runs on Omarchy. It automatically uses the desktop integration while keeping its own name, application identity and `~/.config/maprisma` settings folder. An absolute `XDG_CONFIG_HOME` moves that folder to `$XDG_CONFIG_HOME/maprisma`.

Choose one of these ways:

- **Arch Linux (AUR):** install the package `maprisma` with your AUR helper.
- **AppImage:** download `Maprisma-<version>-x86_64.AppImage` from the releases page at `github.com/uBruckhaus/Asterism`, make it executable and run it. The AppImage uses your distribution's Python 3 with PyGObject, GTK 4 and libadwaita:

Fedora	`sudo dnf install python3-gobject gtk4 libadwaita`
Ubuntu / Debian	`sudo apt install python3-gi gir1.2-gtk-4.0 gir1.2-adw-1`
Arch	`sudo pacman -S python-gobject gtk4 libadwaita`

- **From source:** run `bin/asterism` in a checkout of the repository.

The app brings its own icons, so no icon font is needed. To run agents inside the window, also install VTE for GTK 4 (`vte4`).

2.2 AI Agents Map on Omarchy OMARCHY

Install the plugin from the Omarchy Plugin Marketplace, or from a terminal:

```
omarchy plugin add https://github.com/uBruckhaus/omarchy-ai-agents-map.git --enable
```

A robot button appears in the bar. Everything else it needs comes with a standard Omarchy install. To run agents inside the window, install VTE: `sudo pacman -S vte4`.

2.3 First start

Maprisma's package provides its app-launcher entry. Only the legacy AI Agents Map plugin asks whether to create its own launcher entry.

On Maprisma's first start, if it has no settings yet but the legacy plugin has them, it copies the settings, MCP server list, failed-link records, local-LLM launch settings and agent profiles. A private `plugin-backup-…` folder inside Maprisma's settings folder keeps the original bytes. The plugin's originals remain untouched, and existing Maprisma files are preserved. Verified add-ons for installed agents and local LLMs, plus the online-provider group, are enabled automatically. If Maprisma already has settings, no import takes place.

The window is a normal app window: it can be tiled, floated, moved to another workspace and reached with `Alt`+`Tab`. MAPRISMA A fresh Maprisma opens on its **welcome page**: the agents and local LLMs installed on this computer are cards that are already ticked, and **Switch on** puts them on the map (see Setting up Maprisma). OMARCHY The bar button opens the window, brings it to the front, or hides it when it is already in front. Closing the window hides it; the program keeps running in the background so it opens again at once.

> NOTE While an agent runs inside the window, closing it asks first: **Stop and close** (the agent's local LLM then stops too and frees the graphics card's memory) or **Keep running**.

While the app looks at what is on this computer on the first open, an animated overlay lists what it finds, kind by kind: agents, local LLMs, MCP servers and skills, each with a spinner that turns into ✓ and a number, and how many of them are not connected yet. It fades out by itself after a moment; a click closes it at once. The window opens at once, with this overview as a box in its middle; the map fills in behind it.

2.4 Removing the app

OMARCHY `omarchy plugin remove ubruckhaus.ai-agents-map`. MAPRISMA Remove the package or delete the AppImage.

Before removing, switch off Settings › **Start local LLMs with their agents** so the background watcher is removed too, and Settings › **In the app launcher** to delete the launcher entry. Changes you made with the app stay: agents' settings, MCP servers, skills and installed local LLMs. Uninstall a local LLM first (right-click › **Uninstall…**) if you want it gone. To delete the app's own state as well:

```
rm -rf ~/.config/omarchy/ai-agents-map ~/.config/maprisma \
       ~/.local/state/ai-agents-map ~/.cache/ai-agents-map
```

CHAPTER 3

The window at a glance

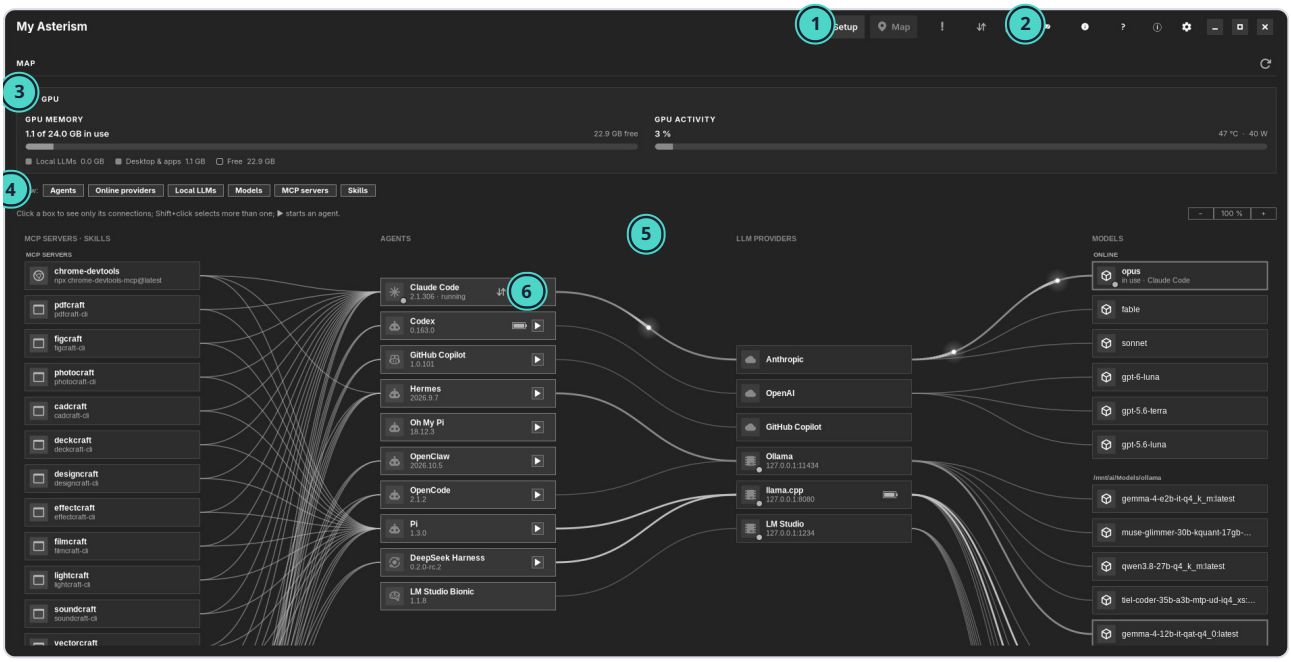


Figure 2 · The main window. The numbers match the list below.

1. **Setup** and **Map** — the two views, on every page; the one in view is highlighted. Setup has every agent, local LLM, MCP server, skill folder and model as a card: install, update, sign in, remove (see below).
2. The header's symbols: ↻ refresh (`F5`: look again, as on start), **!** open problems and updates (red: something to fix; the accent colour: only updates and suggestions; not there when there is nothing), ⓘ the legend (what every dot, line and mark on the map means), ⇅ the network traffic of the running agents (see Network traffic), the two sparkles for **Smart Check** (only while Claude Code or Codex is installed, see Smart Check), the add-ons MAPRISMA, the filled **i** for Information (see Help), **?** Help (`F1`) and the gear for Settings (`Ctrl`+`,`). Hold the left mouse button on a symbol and drag it sideways to put it elsewhere: a bar shows where it lands, and the order is kept.
3. **GPU** — GPU memory and GPU activity: how the graphics card is used right now, in a card that folds (folded, its line keeps the numbers).
4. **Show** toggles — shorten the map to the categories you are working on.
5. **The map** — MCP servers and skills ← agents → LLM providers → models, across the whole width, under its heading **MAP**.
6. ▶ on an agent's box starts it in a terminal in the window; while it runs there, the button brings its terminal back.

3.1 Setup

The map shows what is set up — installed agents and local LLMs, the MCP servers and skill folders agents use, and models — and, faded, the MCP servers and skill folders found on this computer that no agent uses yet, ready to connect with a line (see Found on this PC). Everything else is done on **Setup** (the button in the header, beside **Map**), one card per item, in sections that fold away with a click on their bar:

- **How things get onto your map** — a folded card at the top; open it for five numbered steps: switch on (or find) what you use, install what is missing, sign in to an online agent, connect by drawing, and what shows up by itself.
- **What you can do with this** — what the setup is good for (everyday help, coding, images, voice, research and writing), one tile each with what is there and the next step; folded, its line says how many are ready.
- **AI agents, Local LLMs** — every one the app knows. A card with a blue frame is installed on this PC; a grey one is not, and has **Install** where installed ones have **Uninstall**. OMARCHY A click on an installed card opens its details. MAPRISMA A click ticks or unticks the card (✓), then **Apply** switches its add-on on or off. Switching off takes it off the map; nothing is deleted.
- **Online providers** MAPRISMA — the online providers' add-on; its menu has **Sign in to …** for each installed agent that uses them.
- **MCP servers, Skills, Models** — a card for each one that is installed. A click opens its details (a model: its menu). The card with a **+** at the end of a section adds one: **Add MCP server**, **Add skills**, **Add model**, **Get add-ons**.
- ⋯ or a right-click on a card opens the same menu as the box on the map: update, sign in, start or stop, details, uninstall or remove, move a model.
- A yellow ↑ **version** on a card means an update is waiting; its menu has **Update**.
- A red **!** on a card means an open problem there; hover it for what it is.
- A small battery on an online agent's card shows what is left of its limits (a session, a week): the tightest one counts, yellow at a quarter or less, red under a tenth; hover it for each limit and when it resets. On a running local LLM's card the battery is the GPU memory still free, with the same colours; hover it for how much that local LLM uses.
- The refresh symbol (F5) in the header row looks again at what is installed, set up and signed in, and runs the checks of a start again (new apps, add-ons, updates, open problems).

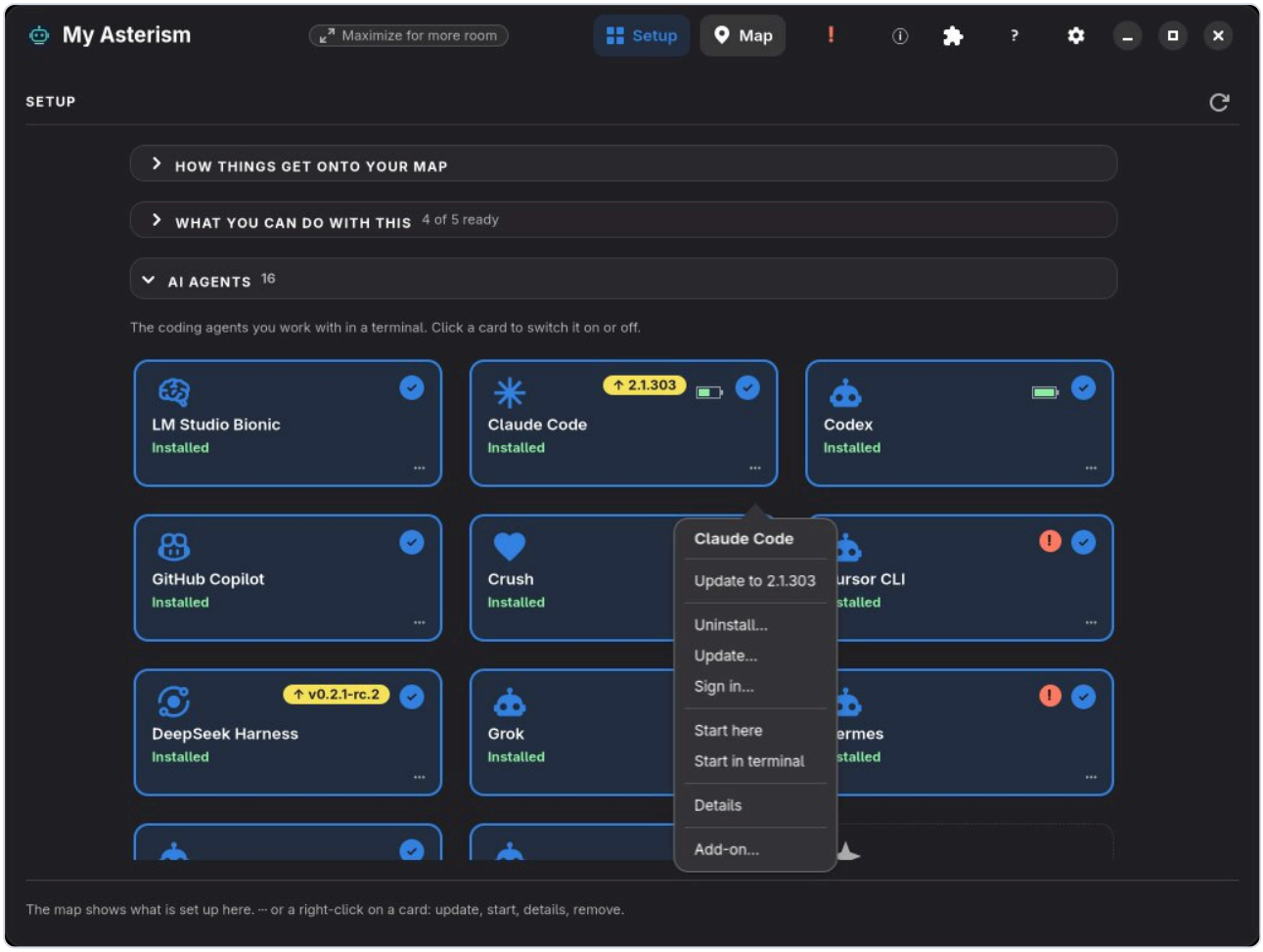


Figure 3 · Setup: installed agents have a blue frame, switched-on ones a ✓, updates a yellow and problems a red badge; the menu of Claude Code.

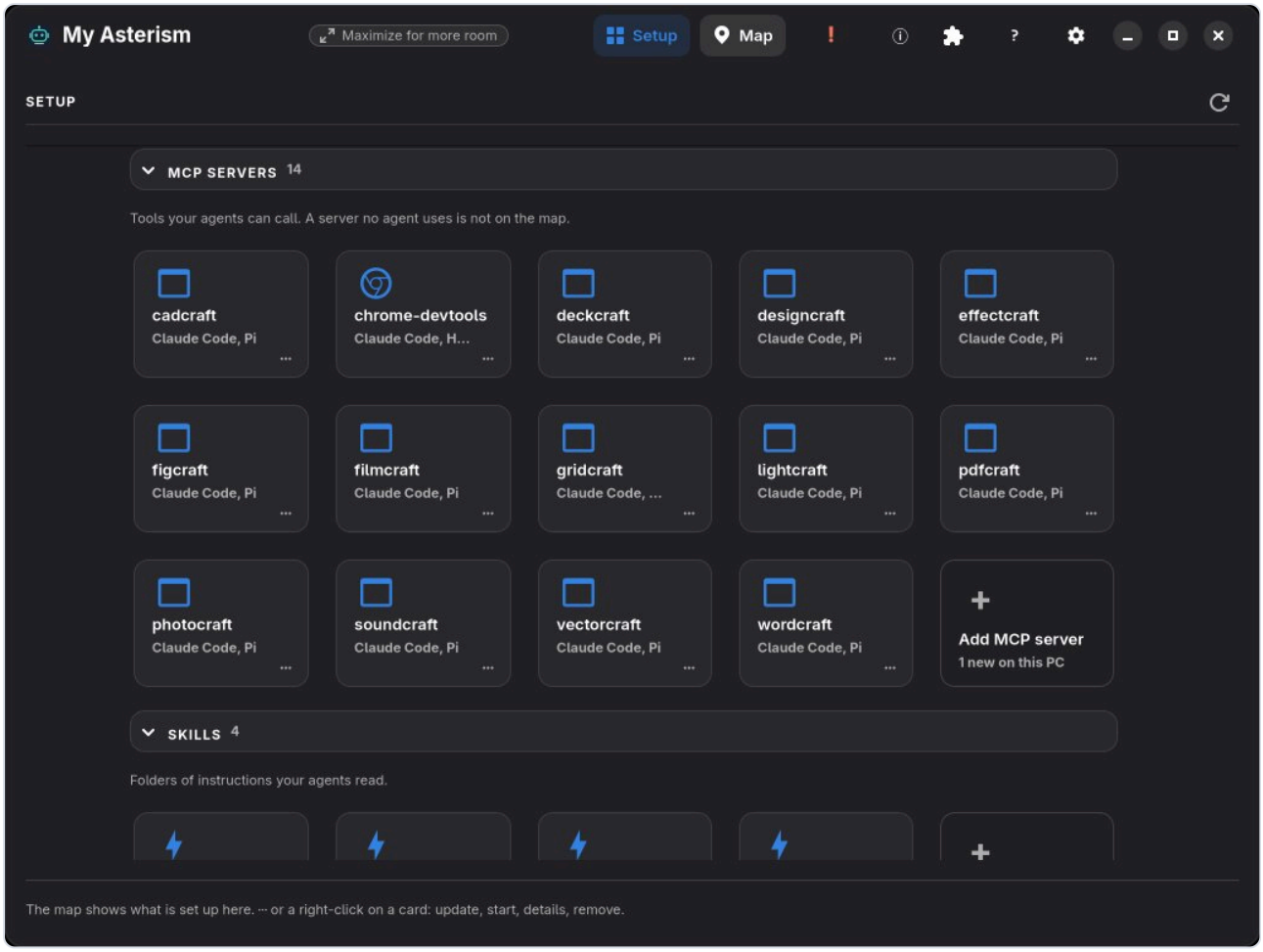


Figure 4 · MCP servers and skills, each section with a card to add one.

There is nothing to hide: what you do not want on the map is switched off or removed here. When an install or a sign-in runs in a terminal in the window, **Back** (or **×** on its tab) returns to the page it was started from.

Local MCP servers and **Online MCP servers** (those reached at a web address) are two sections. **Add folder...** in **Local MCP servers** and in **Skills** adds a folder where the app looks for them too — for example a shared disk with unpacked apps that offer an MCP server, or a folder of your own skills. The folders added are listed beside the button; **×** on one stops looking there (nothing in it is deleted). What is found in them shows on the map, faded, until a line connects it: Setup only has cards for what is set up.

3.2 Choosing what the map shows

The map shows everything that is set up. The **Show** toggles below the GPU card leave whole categories out of it — for example all skills, or the local LLMs and their models — and are remembered.

In a narrow window (half the screen or less) the map chooses them itself, and a line beside the toggles says so: agents and providers (who works through what). A toggle clicked there stays until the window is full size again and is not remembered; the full-size window keeps your own choice. More categories switched on do not squeeze the boxes: the map scrolls sideways.

3.3 Working with the map

- **Hover** a box to light it up together with everything it is joined to: their frames turn the accent colour, the rest of the map dims. Otherwise every box has the same thin grey frame.
- **Click** a box to select it (✓); `Shift`+click adds more. The selected boxes and everything joined to them keep the accent frame while the pointer is not over another box. `Esc` or **Clear selection** clears it.
- The **pin** at a box's top-right corner keeps its connections lit while you look elsewhere.
- **Right-click** a box for everything its card on Setup offers: start or stop, details, install, sign in, models, update, remove, and more.
- ▶ on an agent's box starts it in a terminal in the window.
- The agents always stay in the middle of the map's visible part while you scroll.
- Each agent's active path — agent → provider in use → its model — is drawn bold. While an agent is working, light runs along that line.
- The **usage statistics box**: while an agent is working, small numbers are written along its line to the provider, in the line's colour: how fast its latest reply came (**t/s**, tokens per second), its context size (**1M**) and how much of it is in use (**%**, yellow from 75 %, red over 90 %, like the batteries). They come from the agent's own session files (numbers only; Claude Code, Codex and Pi). What it shows, and its text size, are in Settings.
- A red dotted line joins an agent to a provider it is set up for but does not use now (no model of it in use).
- When agents use different models of one local LLM, the lines to those models are violet: it keeps them loaded at once, or swaps them (see Models at once).
- An online agent's box has the same battery as its card on Setup: what is left of its limits; a running local LLM's box, the GPU memory still free.
- An agent paid per token — signed in with an API key, so it has no plan limits for a battery — shows instead what its sessions active today cost at API prices, as a small **"$3.40 today"** on its box and card: green, yellow from 75 % of the day's limit and red over it (Settings › **Costs**), refreshed every minute. Its details show the whole API cost.
- When a program that is not on the map uses a local LLM, the line from that LLM to the model turns bold in a colour of its own (purple or cyan, whichever stands out from the theme's other colours), and light runs along it. See Used by other apps.
- When an agent started another one — Claude Code running `pi -p …` or `codex exec …` through its shell, for example — an **arrow** runs along the left of the agents' card from the agent that uses the other to the one it uses, with light running along it; its tooltip names both. An agent can use several others at once, each with an arrow of its own. While it is used, the agent it uses has its lines to its provider and on to its model in the arrow's colour too, and the light on them runs in that colour: you see at a glance which provider and model work for whom.
- The refresh symbol (`F5`) in the header row looks again at what is installed, set up and signed in, and runs the checks of a start again (new apps, add-ons, updates, open problems).
- Until the first line is drawn, a card above the map shows how, in three steps; **Watch how** plays the animation from Help. It goes for good after the first connection, or with ×.
- Zoom with **− 100 % +**, `Ctrl`+scroll or `Ctrl`+`+` / `-` / `0`.

- The boxes sit in one card per category (agents, online providers, local LLMs, a card per model folder, MCP servers, skills). Drag a card by its header to put it anywhere on the map, and a box inside its card to change the order.
- **Click a card's header** to fold the card to its header (▸ at the header's right) and again to open it (▾). The lines of a folded card's boxes end at its header, so you still see what is joined to it (the arrows between agents hide while the agents' card is folded). Opening a card moves the cards you put around it out of its way.
- The **pin** right of the zoom keeps the map's layout — where you put the cards, the order of the boxes and which cards are folded — after **Refresh** (F5) and a restart. Unpinned, as it starts, the map comes back in its default layout on every refresh and start; your changes last until then. **Default layout** puts everything back at once.
- **Faded boxes** in the MCP servers and skills cards are on this computer but used by no agent. Drag a line from an agent to one to connect it (see Found on this PC). MCP servers reached at a web address have a card of their own, **Online MCP servers**, beside **Local MCP servers**; both fold like any other card.

> **TIP** In a narrow window (below about 1300 pixels, such as half the screen) the map scrolls sideways.

3.4 Open problems

When something needs fixing, a red **!** appears among the header's symbols; its tooltip says how many, for example *2 open problems · 4 suggestions*. A click opens **Open problems**: one line each — an agent not signed in, a local LLM without models, a connection that does not work, little free space for models — with a button to where it is fixed (the agent's or local LLM's details, the map with that line lit, Setup, Tidy up). The same problems show as a red **!** on their cards on Setup. Tips and what works are on the cards and details pages themselves, and what the setup is good for is on Setup (**What you can do with this**). The check only reads; it never changes, starts or sends anything, and it shows nothing while the map is still empty.

3.5 Open problems in the Flutter preview

In the Flutter preview, a **!** in the header (red when something needs fixing) opens **Open problems**: each thing to fix — about this PC (free space for models), the agents, the local LLMs and models, or the tools — with a button that opens where it is fixed. **What you can do with it** shows the use cases (everyday help, vibe coding, images, voice, research and writing) as **Ready**, **Almost** or **Not yet**, with what is there for each and the next step. **Check again** looks once more. The checks read only what the app already knows plus the graphics card, the memory and the free disk space; nothing is changed. **Updates and suggestions** lists newer versions of the installed agents and local LLMs, each checked the way it was installed: mise's outdated tools, an agent's own release channel (Claude Code's stable or latest), and the local LLMs the app installed itself. **Update** runs `mise upgrade` or the agent's own updater in a terminal of its own, or opens the install review with the new release; the web is asked at most every six hours.

3.6 Updates and suggestions

Below the problems on the same page, **Updates and suggestions** lists what there is to say: new programs or skills found on this PC, what is on this PC but not on the map, and updates for installed agents and local LLMs, each with its button (**Add MCP server…**, **Setup**, **Update**). With only these and no problem, the header's **!** shows in the accent colour. Updates are checked in the background when the window opens, at most every six hours. Nothing is installed by the check itself; an update starts only when you click **Update**.

With Settings › **Daily AI checks** on, the MCP server updates an agent found on the web are listed here too, with **Release page** and **Do it with …**; once the new version is on this PC, the line offers **Use it…** instead (see Updates found).

3.7 Add-on suggestions in the Flutter preview

In the Flutter preview, the app reads the signed add-on catalog in the background at start: from its cache, and online only when **Look for add-ons online** is on. Open problems then lists under **Updates and suggestions**: agents and local LLMs on this PC whose add-on is switched off or not installed yet (**Get add-ons** or **Add-ons** opens the add-ons to switch on or get), and installed add-ons with a newer version in the catalog. **Update all** shows them with their versions and, once confirmed, updates them one after the other, each checked against the catalog; one message says what was updated and what was not.

CHAPTER 4

Agents

4.1 Agent details

Every coding agent has a card on Setup, installed ones with their version. Click an installed agent's card (MAPRISMA or ··· › **Details**), or right-click its box on the map › **Details**. At the top, one line says whether the agent is ready (and if not, why) and whether it is running now or when it was last used. Below it, every category is a card that folds: click its heading to open or close it. They start folded, each with a one-line summary and badges; a category you open stays open on the next details pages.

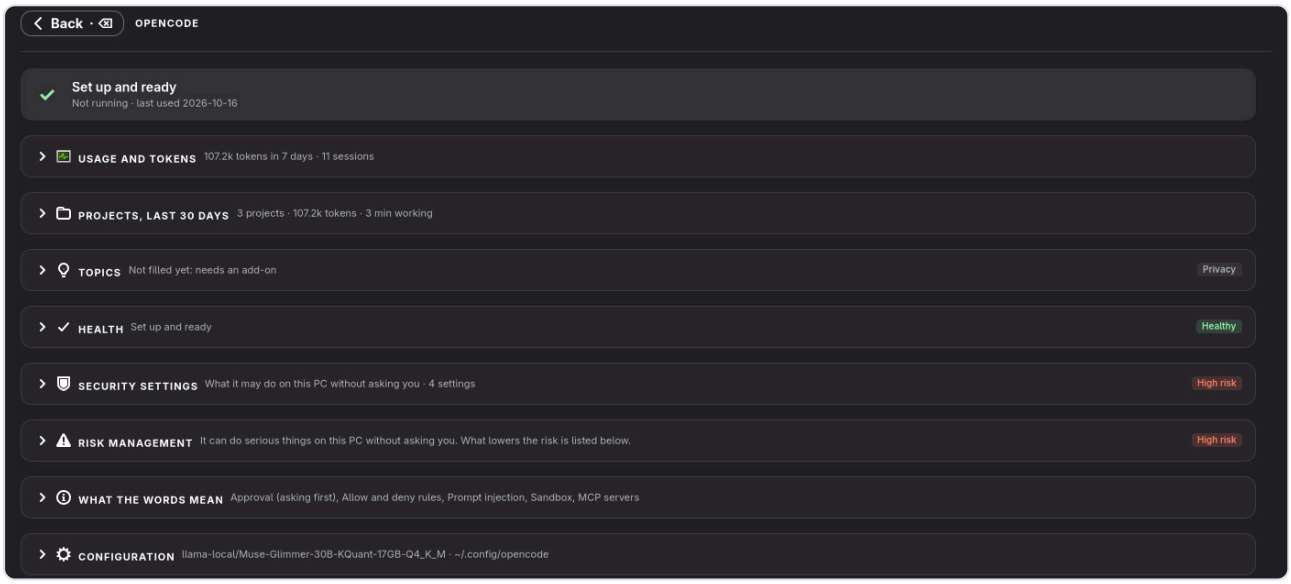


Figure 5 · An agent's details: every category folded, with its summary and badges.

Card	What it shows
Usage and tokens	Where the agent records it: today's and the last seven days' tokens and sessions, the plan, a chart of tokens per day, rate limits with reset times and tokens by model.
API cost / **API value**	What its tokens cost at the providers' API prices: today, the last 7 and 30 days, and per model the split into input, output, cache read and cache write (tokens and amount). **API cost** for an agent paid per token (with its share of the day's limit), **API value** for one on a plan (the plan covers it). See API cost.
Projects, last 30 days	Per project folder: working time, sessions, tokens, subagent runs, lines changed and the API value.
Topics (Privacy)	What the sessions were about. It goes beyond numbers, so it is marked **Privacy** and filled only by an add-on (see below); the add-on names the card and its badge says how much it reads.
Health	Every setup check with a ✓, or what to do next: sign in, install or start its local LLM, download a model, add MCP servers or skills.

Card	What it shows
Security settings, **Risk management**, **What the words mean**	What the agent may do without asking you, and what lowers the risk (see Healthy & Security check).
Configuration	The agent's settings, MCP servers, skills and settings folder.

Projects are shown for every agent whose sessions the app reads — Claude Code, Codex, Pi, Oh My Pi, OpenCode, Hermes, DeepSeek Harness, Goose, Gemini CLI and OpenClaw: working time, sessions, tokens, subagent runs, lines changed and what the tokens would cost at API prices. The numbers come from each agent's own session files or database; no text is read.

API cost. The amounts are an estimate: the token counts the agent keeps (input, output, cache read and cache write, per model) at the providers' published API prices — Anthropic, OpenAI, DeepSeek, xAI and Google, with the date they were taken. A session counts on the day of its last reply; local models cost nothing. Crush keeps no token counts that add up, so its card shows the cost Crush itself reports. For an agent that keeps no token counts on the PC (Cursor CLI, Antigravity, Muse) the card says the cost is not known; its provider's account page has it. Amounts are shown in US dollars, or in euros at the European Central Bank's daily rate (Settings › **Costs**).

MAPRISMA With the **Insights** add-on (see Add-ons) the **Topics** card shows first which topics took the most tokens in the last 30 days (**Most tokens**: sessions with the same title are one topic, each with its share of all tokens), then the titles the agent gave its sessions itself, per project (**By project**), and offers **Summarize with Claude…** (when Claude Code is installed). Its tokens come from the agent's own session files without cache reads, so they can be far lower than in **Usage and tokens**, which may count cache reads. Before anything is sent, you see exactly what will be sent: the titles and numbers, never the conversations.

4.2 Starting and stopping agents

The last 50 project folders are kept per agent.

Each agent’s **Start** submenu has three sections separated by lines: **Start** with the current folder, **Choose folder…**, and a searchable **Recent projects** drop-down. The drop-down scrolls and lists all folders used to start this agent from the map, newest first. Choose a past project, then Start. Choosing a folder in the folder picker starts one session there. No pinned folder is required: without one, Start uses Settings › **Agents start in**, or your home folder. The agent’s details page has a scrollable **Recent projects** card with every recorded folder. Click a row to open Start with that folder selected. Its pin sets this agent’s next default; only one folder per agent can be pinned, and unpinning restores the general default. The trash button removes the entry and its pin from this agent’s history, never the folder or its files. The same submenu, history and pins apply to **Start here** and **Start in terminal**. **Start again** offers the submenu again. Running sessions keep their existing folder; cancelled choices and failed launches add no history.

- ▶ on the agent's box, or **Start here** in its menu, runs the agent in a terminal inside the window, in place of the map. This needs VTE (`vte4`).
- **Start in terminal** opens it in a terminal window of its own.

- While it runs inside the window: **Show terminal** and **Stop**. Stop ends the agent and everything it started, such as its MCP servers.
- **Terminals…** lists where else the agent runs: *Here, in this window*, each *Terminal window* (a click brings it to the front) and each instance *In the background* — one with no window, such as `claude -p …` run by a script or by another agent. A click on a background instance opens a terminal tab that shows what it writes, as it writes it: questions, answers and the tools it uses (from its session file, or its output when that goes to a file). The tab ends when the instance ends. This is the one place where the app shows conversation text, and only for the instance you click.
- **Stop…** — also the small stop button left of ▶ on the agent's box — stops one of its instances outside this window. Pick it from the list, which says where and when each one was started. It is asked to end (SIGTERM) and, if it is still there after 3 seconds, ended. Only that very process is stopped: if it has ended in the meantime and its process number now belongs to another program, nothing happens.

When agents have terminals in the window, a tab line above the map shows one tab per terminal with a dot for *working*, *running* or *ended*. Click a tab to show that terminal; **×** closes an ended one.

An agent with an open instance anywhere on the computer gets a green dot and *running*, and the model box it runs with says *in use · <agent>*.

4.3 Network traffic

A running agent's box and card show what it receives and sends (↓ in, ↑ out). Click that line, or ⇅ in the header for all running agents, to open its network page: **Now**, **The last 5 minutes** as a chart (in green, out blue), **Where it goes** — online APIs, local LLMs, this PC and your local network, each with its share — and **Programs**: the agent itself and the programs started by it, such as its MCP servers. It is counted from the agent's own TCP connections while the app is open (no administrator rights needed; UDP is not counted) and stays on this PC.

4.4 Subagents

Claude Code and OpenCode can hand parts of a task to **subagents**. While one of an agent's subagents works, its box on the map shows a small tree mark next to ⇅; while it has only finished ones listed, the mark is dimmed. Click the mark to open the agent's **Subagents** page: a map of its own, laid out like the big one, with the agent's card at the left and a **Subagents** card at the right. Each box shows the subagent's task, its type, the folder its session works in, and **working** or **done … minutes ago**. Light runs on the lines to the working ones; finished ones are dimmed. Hover a box to read its whole task.

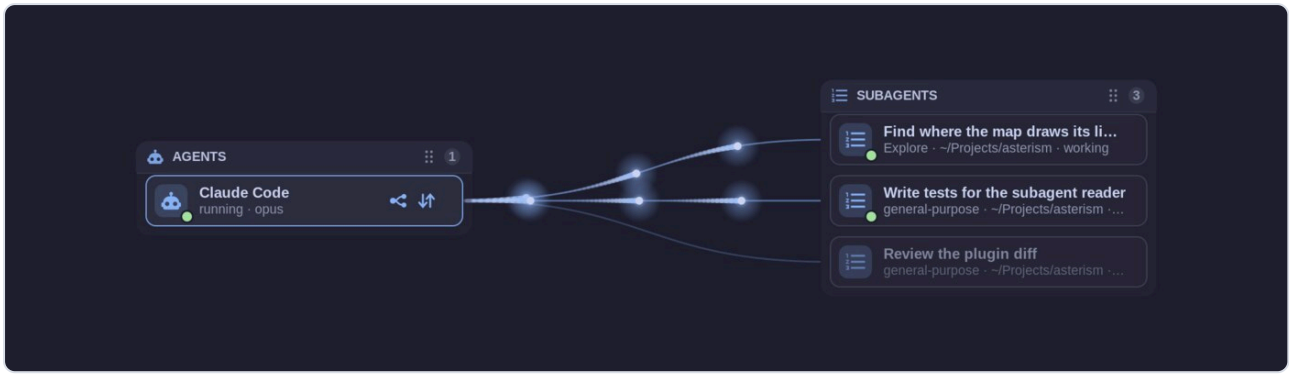


Figure 6 · An agent's subagents: two at work, one finished.

How long finished subagents stay listed (30 minutes) and how many at most (10) is set in Settings › **Subagents**; working ones are always shown. The page reads only each subagent's type and the one-line description of its task, from the agent's own files (Claude Code's session folders, OpenCode's database), never what the subagents say.

4.5 Agents working together in the Flutter preview

In the Flutter preview, an agent that started another one and uses it — Claude Code running `pi -p …` through its shell, for example — is joined to it on the map by a line bent along the right of the agents' boxes. Light runs on it while either of them works. It is found from the process tree only (the agents' own processes and their parents), and the line is shown, never edited. The subagents of Claude Code, Codex, GitHub Copilot, OpenCode and Goose are read as in the app.

4.6 Costs in the Flutter preview

An agent's details in the Flutter preview show what its tokens cost at the providers' API prices: today, in the last 7 and 30 days, and per model split into input, output, cache read and cache write. **API cost** for an agent paid per token (with the share of the day's limit), **API value** for one on a plan (which covers it). It is an estimate from the numbers in its own session files; no text of them is read. Settings › **Costs** sets the **Currency** (euros at the European Central Bank's daily rate, fetched once a day), **A day's limit** and the levels **Yellow from** and **Red from** (red always above yellow). Claude Code's session files are read on from where the last look stopped, so very long sessions are counted whole; a first look on a PC with very many of them may take two looks to be complete.

On the map, an agent with plan limits shows a small battery in its box with what is left of the tightest limit, and one paid per token shows today's cost there instead; green, yellow or red at the levels of Settings › Costs, the details in its tooltip. Both are looked at again every minute. With **Notify when red** on, a cost in the red gets one desktop notification a day.

4.7 Details in the Flutter preview

An agent's details also show **Projects, last 30 days**: per project folder the working time, sessions, tokens, lines changed and API value, the subagent runs per type and the busiest hour, from the numbers in its session files. A local LLM's card says whether it runs, how much GPU memory it holds and which models are loaded, shows its models, agents and address, **Usage and tokens** that agents

sent through it (matched by model name), and under **Risk management** who can reach it. On the map, a running local LLM shows the GPU memory still free as a battery. Clicking an MCP server's or a skill set's box opens its details: which agents have it and how each starts, calls or reads it (values of keys, tokens and query parameters are never shown), who has not got it yet, and the server's tools as last asked or each skill's description; **System Map** opens the system map as before.

4.8 Terminal tabs, the usage box and the starter model in the Flutter preview

While agents run in terminals in the window, a tab for each sits above the map, with a dot for **working**, **running** or **ended**; a click shows the terminal, the one in view is marked (Settings: **Terminal tabs above the map**). A working agent's box on the map shows the **Usage statistics box**: how fast its latest reply came (tokens per second), its context size and how much of it is in use (yellow from 75 %, red over 90 %), read from its own session files, numbers only; Settings says which of these show and how big. In **Your first agent**, a route with a local LLM suggests the starter model that fits your graphics card (**A model**), and **Download it** starts its download as the Models page does.

4.9 Configure…

Right-click an agent or a local LLM › **Configure…** opens its own setup in a terminal tab of the window, for what the app does not set itself — for example `hermes setup` or Ollama's own menu; LM Studio opens its window. A program without a setup command shows its `--help`. When it ends, the map looks again and shows what changed, and a running local LLM is offered a restart.

4.10 Signing in

Sign in… in an agent's menu (on Setup or on the map) shows what the agent needs and its own ways to sign in:

- **A subscription or an account** — the agent's own sign-in runs in a terminal in the window (for example `claude auth login` or `codex login`). Your password or browser login stays between you and the agent; the app never sees it. Where the agent asks for it, the terminal says what to type (Pi and Copilot: `/login`).
- **An API key** — for agents that read one: the key goes into a password field, then straight to the agent's own command (Codex stores it itself) or into the file the agent reads (Gemini: `~/.gemini/.env`, readable only by you). It never goes into a command line, a log or the app's settings, and the app does not keep it. Claude Code's API billing is its own Anthropic Console sign-in, so no key passes through the app.

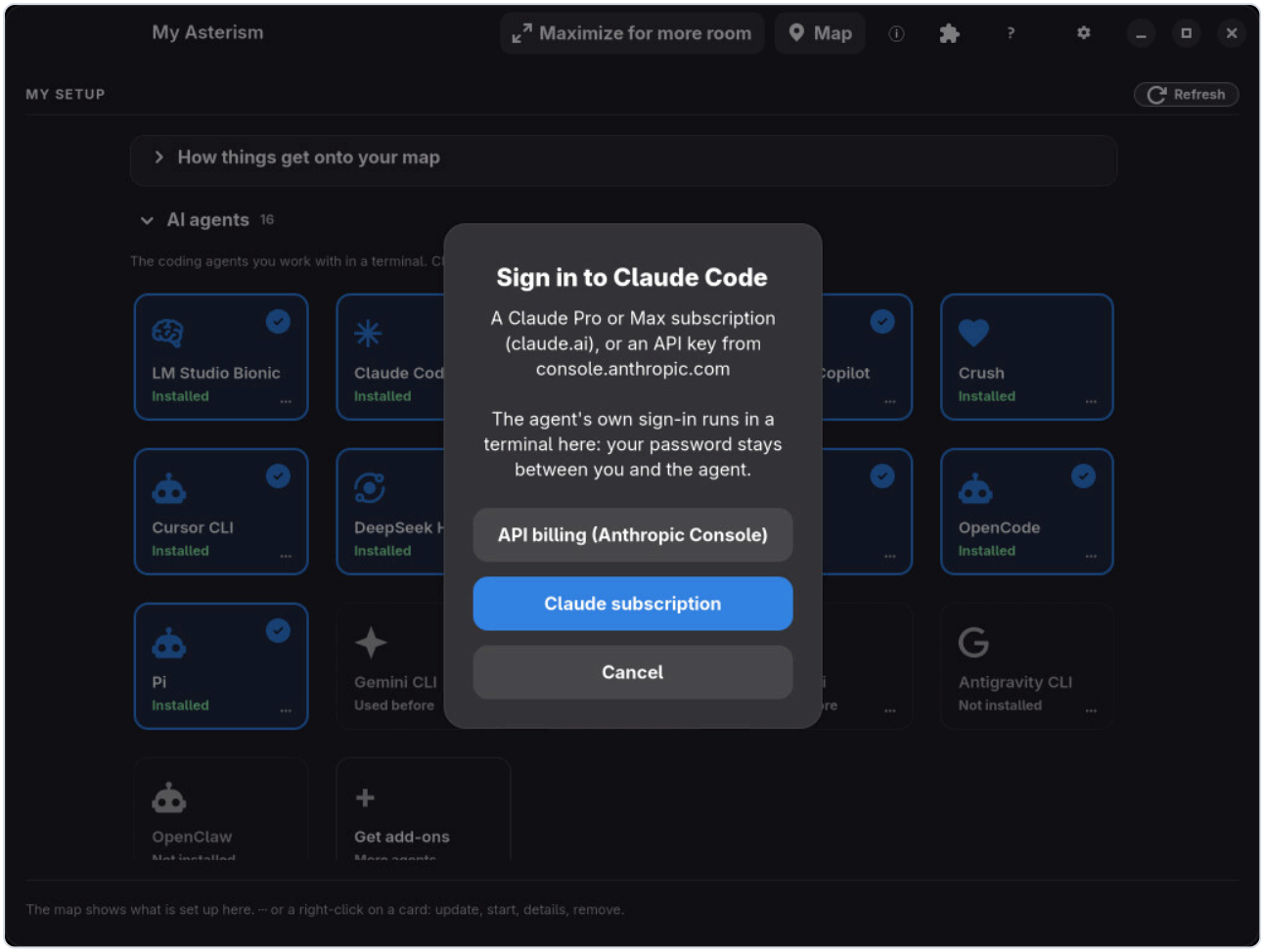


Figure 7 · Signing in to Claude Code: a subscription or API billing, both with Claude Code's own sign-in.

Afterwards the map shows the agent's online provider and its models.

4.11 API keys in the Flutter preview

In the Flutter preview, an agent that reads an API key shows **API key…** in its details. The card says where the key goes before you type it: to the agent's own command (Codex: `codex login --with-api-key`), or into the file the agent reads (Gemini: `~/.gemini/.env`, written as a new file readable only by you; other lines in it stay). The field hides what you type, and **Save** becomes available only for something shaped like a key. The key goes to a separate worker process on its standard input; it never goes into a command line, a log, a backup or the app's settings, and the app does not keep it. When the agent's own command answers, its last line is shown, with the key cut out.

4.12 Updating agents

When a newer version of an installed agent is out, its card on Setup gets a yellow ↑ **version**, and **Updates and suggestions** (the header's **!**) names it. **Update** runs the update the same way the agent was installed — for example `mise upgrade <tool>` or `claude update` — in a terminal window of its own, so you can follow it.

4.13 Updating a running agent in the Flutter preview

When you click **Update** for an agent that is running right now — in a terminal tab here or in a terminal of its own — the Flutter preview first says so: an update replaces the agent's program while it works, so its sessions may stop or behave oddly, and some updaters refuse. **Cancel** leaves everything as it is; finish the agent's work and update later. **Update anyway** starts the update in a terminal as usual.

4.14 Agents the app can install

Four agents can be installed, updated and uninstalled from the map (right-click the agent › **Install…**):

- **Goose** (`goose` , from the Agentic AI Foundation, formerly Block), its GitHub release through mise. It works with many providers and with local LLMs; **Configure…** runs `goose configure` .
- **DeepSeek Harness** (`dsh` , developer preview), from npm. It can use local LLMs.
- **Antigravity CLI** (`agy` , Google), from Google's release manifest; the download is checked against its SHA-512.
- **LM Studio Bionic** cannot be installed from here (no published checksum); get it from lmstudio.ai. It always uses LM Studio's engine and models.

4.15 New agents

When an agent turns up that the app does not know yet, it asks an installed agent (Claude Code, else Codex) once to find out on the web where that agent keeps its skills and MCP servers. The answer is checked before use, and nothing in it is ever run. Settings › **Look up new agents** switches this off.

CHAPTER 5

Local LLMs

A local LLM runtime runs language models on your own graphics card, so prompts never leave the computer. The app installs, starts and stops them for you.

5.1 Installing a local LLM

Right-click a grey (not installed) local LLM on the map › **Install…**. You choose the program folder, the model folder (the one your other local LLMs use is suggested) and whether it starts at login. It is installed for your user only, with a user service, so no password is needed. Every download must match the SHA-256 checksum its release publishes before it is unpacked or run.

Runtime	Notes
Ollama	The official release (with ROCm for AMD cards).
llama.cpp	The build of your choice (Vulkan, ROCm, CUDA, CPU), run as a model router: every GGUF in the model folder, one loaded at a time.
LM Studio	Not installed from here (no published checksum); get it from lmstudio.ai. Shown and managed once installed.
vLLM	For NVIDIA cards, in its own Python environment.
Jan	Its AppImage, unpacked, with Jan's engine as a background service for agents.
KoboldCpp	The official single-file build, run as a model router.

Installed this way, a local LLM also offers **Update…** (the previous version is kept as a fallback), **Model folder…** and **Uninstall…**, which removes the program, command, service and settings but never the models. Local LLMs you installed yourself are shown, but never changed.

5.2 Installing in the Flutter preview

In the Flutter preview, a local LLM that is not installed yet shows **Install** under Providers and models, and the first-agent guide offers **Install mise**, **Install Node.js** and **Install** for the local LLM its route needs. Maprisma first looks up the latest release and shows it on a card: its files and their size, the program folder (`~/.local/share/ai-agents-map/<name>` , one folder per version), the model folder inside it and your graphics card. For llama.cpp and KoboldCpp, choose the build there (Vulkan, ROCm, CUDA or CPU); another build is looked up again before anything is installed. A local LLM with a user service also offers **Start now** and **Start at login**.

Install downloads the files, checks each one against the checksum its release lists, and unpacks them; nothing of the download runs while it is installed (Jan’s AppImage unpacks itself in a closed-in sandbox). A program that is already installed some other way, or a settings file or service that already exists, is never replaced: the card refuses it. Ollama, llama.cpp, KoboldCpp, Jan and mise install this way; LM Studio and vLLM do not, because no checksummed release can be taken for them.

Every step you start from the first-agent guide — installing, signing in, the test question, Providers and models, Connections — brings you back to the guide with **Back**, refreshed, so the step just done shows as done.

5.3 Starting and stopping with agents

Connect an agent to a local LLM once, then just start the agent — even with the app closed or after a reboot. Its local LLM starts with it, and when the last agent using it closes, the LLM stops about five seconds later, so its graphics memory is free again.

A small background service, `ai-agents-map-providers.service`, does this. It is set up with the first connection to a local LLM and removed when the last one goes, or when you switch off Settings › **Start local LLMs with their agents**.

> **NOTE** KoboldCpp takes a few seconds to start, longer than some agents wait. So it starts at login, and when the last agent closes only its model is unloaded.

5.4 A local LLM's details

Click a local LLM's card on Setup, or right-click its box on the map › **Details**. Below the tiles (models, agents, address) the same folding cards as for agents: **Usage and tokens** — what your agents sent through it in the last days, per agent and model, taken from the agents' own session numbers and matched by model name — then Health, Risk management, What the words mean and Configuration.

5.5 Performance

A local LLM whose settings the app knows (llama.cpp, Ollama installed by the app) has a **Performance** card on its details page:

- **Build**, **GPU stacks** and **Video memory**: what runs it — ROCm, Vulkan, CUDA or the CPU (for llama.cpp read from its own device list, also for a build you made yourself), which of these stacks this PC has, and the card's memory.
- **Context per model**: each model's file size, the context set now, the context the card holds for it — its file plus the context's cache, read from the model's header — and what it was trained for. **!** marks a context larger than the card holds (the model would not load, or spill into RAM), ↑ one the card could make larger; **Set to …** changes it.
- **Ask …**: the local helper (or the online agent) looks at these facts — names and numbers only — and suggests changes: settings such as flash attention, the context's cache type, the GPU layers, parallel requests and batch sizes; contexts; another build when it would clearly help. The app keeps only suggestions it can stand behind (a known setting with an allowed value, a context the card holds) and shows each with why. **Apply** writes it with a backup; the message offers **Undo**, or **Restart now** when it counts only after a restart. Another build is only suggested: install it from the local LLM's menu.

For Ollama, its settings are for every model at once (its server's environment file).

5.6 Performance in the Flutter preview

In the Flutter preview, a local LLM with settings the app can tune (llama.cpp, Ollama) has **Performance** in **Providers and models**. It shows the build it runs as, the GPU stacks on this PC and the video memory, then **Context per model**: each model's size, its context now, what the card holds for it and what it was trained for. A model with more context than the card holds is marked, and so is one the card has room for; for llama.cpp, **Set to …** sets it. **Settings for every model** lists the engine's settings with what each does; one with a list of values can be changed there. Every change is shown first and written with a backup, so **Undo** puts it back; it counts after the local LLM restarts. Under **Tips from an agent**, **Ask …** has the agent for AI checks look at these facts (names and numbers only, through the web alone): it suggests settings, contexts, another build or notes, each with why. The app keeps only what it can stand behind — an allowed value, a context the card holds, a build the local LLM has — and **Apply** goes through the same preview. Its tokens are the agent's; the run is listed in the Smart Check.

5.7 GPU memory and Free GPU memory

In the **GPU** card above the map, **GPU memory** shows how much is in use, split into local LLMs, desktop and apps, and free; **GPU activity** shows load, temperature and power. Each local LLM's box shows how much memory it holds, and loaded models get a green dot.

Right-click a local LLM for **Start**, **Stop**, start at login, and **Free GPU memory**: its models are unloaded, it keeps running and loads one again when asked.

When the last agent that used a local LLM closes and the LLM still holds its models, a message asks whether to give the memory back: **Free it** unloads them, **Keep** leaves them loaded.

A local LLM that holds GPU memory gets a red stop button on its box. It asks first: **Free GPU memory** unloads its models; **Stop** ends it. For LM Studio the button is **Stop server**: only its server ends, and the app stays open with its models loaded. A model that a local LLM has loaded, but no agent uses, is joined to it by a solid light line.

LM Studio can be open without its server. Its box then says *server off* with a yellow dot, agents cannot reach it, and its menu offers **Start server** (`lms server start`).

5.8 Models at once

llama.cpp, LM Studio and Ollama can keep more than one model loaded. When you connect agents to different models of one local LLM, the app sets how many it keeps loaded at once to the number its agents use — with the same preview, backup and Undo as the connection — and offers to restart it if it is running, because it reads the number when it starts. Where that number is not the app's to change (a system install, or your own start command), a note says where to set it.

When the models together do not fit in the GPU's memory, a warning says so: the local LLM then swaps them, which is slow, or part of a model runs on the processor. Starting an agent checks the same: if loading its model needs more GPU memory than is free, you are asked first, with what holds the memory now — **Free GPU memory, then start**, **Start anyway** or **Cancel**.

5.9 Models at once in the Flutter preview

In the Flutter preview, after a change of an agent's local LLM or model is applied, the app looks at how many different models each local LLM's agents now use. Where it may set the number — llama.cpp's and Ollama's settings files of the app's own installs, LM Studio's one-or-many setting — a second **Preview** shows the change, with what it means and any warning; **Apply** writes it with a backup, and **Undo** takes it back. A number fixed in your own start script is left as it is. When the models are too many for what is kept loaded, or too big together for the graphics card, Open problems says so as well.

5.10 Used by other apps

Local LLMs are not only used by agents. A plugin such as Video Trans, a chat app or your own script can send requests to them too. The app notices this and shows it:



Figure 8 · A program that is not on the map (here `curl`) generating with llama.cpp: the cyan line and box (Tokyo Night theme). Claude Code's own path to opus stays in the accent colour.

- **On the map**, the line from the local LLM to the model it is generating with turns bold in its own colour, light runs along it, and the model box gets a dot in that colour and says who is using it, for example *in use · Video Trans*.
- **The local LLM's right-click menu** and its ⓘ **Details** page say *In use by Video Trans · now*, or how many seconds ago it was last used.
- The ⓘ legend shows both marks.

The app looks at which of your own programs have a connection open to the local LLM. It never reads what they send. A plugin is named by its own name, an agent by its name, anything else by its program name. The app itself and LLM Monitor, which only read the status, are not counted.

> **NOTE** The light shows only while the program is generating, and fades about ten seconds after its last request. A program that is open but idle — Video Trans with no speech to translate — shows nothing. A program stays listed in the menu for half a minute after its last request.

CHAPTER 6

Models

Right-click an installed local LLM › **Models…**, or click **Add model** on **Setup**. The page has a list on the left and the chosen model on the right, in three tabs.

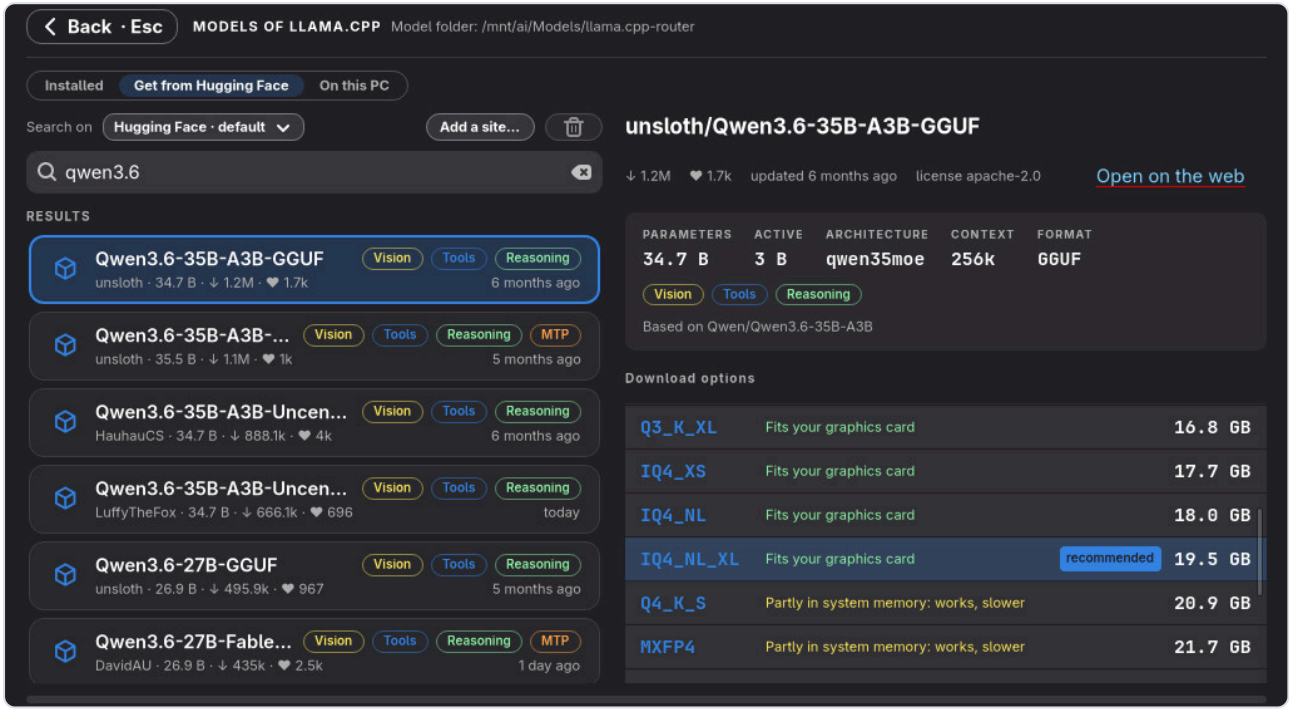


Figure 9 · Get from Hugging Face: what a model can do, and which size fits your graphics card.

6.1 Get from Hugging Face

Browse the trending GGUF chat models or search. Each result shows its author, size, downloads, likes, age and what it can do: **Vision**, **Tools**, **Reasoning**, **MTP**. The model's page shows its parameters, architecture, context, license and model card. **What the name means** explains the parts of the name, such as *35B-A3B* or *Q4_K_M*.

Download options lists every size with how it fits: in the graphics card, partly in system memory (works, but slower), or likely too large. The best fit is chosen for you; **Download** gets it into the model folder. For Ollama, which downloads by the size's name, only the sizes it can download are offered: a split file, one in a sub-folder or one Hugging Face refuses for Ollama is dimmed, with the reason, and never recommended.

Downloads run in the background and show in a strip at the top of the main page. A broken connection is retried and the download resumes. If the app was closed during a download, it is listed as interrupted, with **Continue** or **Discard**.

6.2 Interrupted downloads in the Flutter preview

In the Flutter preview, a model download or an Ollama pull whose connection breaks off is tried again after 5, 15, 30, 60 and 120 seconds and goes on where it stopped. If the app ends before it does, the next start shows a banner with each interrupted download, where it stopped, **Continue** (it goes on from the part already downloaded) and **Discard**. Only how to start it again is kept, in `~/.local/state/maprisma/jobs.json`, readable only by you; a download that is still running elsewhere is not started twice.

6.3 Ask an agent which model fits

Not sure which model to take? When an agent for AI checks is set up (Settings › **Agent for AI checks**), the **Ask an agent** card under the search takes a question in your own words — for example *a coding model with a long context that fits my graphics card*. **Ask** (or `Ctrl`+`Enter`) sends it, together with your graphics card, its memory, the computer's memory and the models you already have, to that agent, which looks on the web; a spinner and the time taken so far show while it works. After a minute or two the card shows its short summary, and the models it suggests appear in the result list below, headed *Suggested by …*: each found on Hugging Face, with its size and why it fits, opening its page and download options on a click like any search result. A suggestion Hugging Face does not have is named under the heading. It suggests up to 10 models; **Suggestions**, at the right of the agent line, sets another number (5, 10, 15 or 20). It uses the agent's tokens; the question is not kept, and each local LLM's page asks on its own.

6.4 Installed

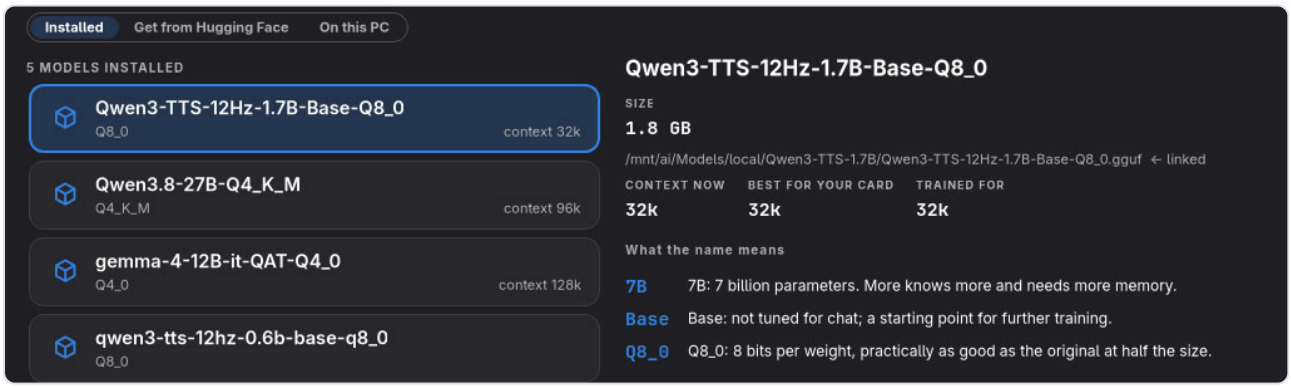


Figure 10 · Installed: context now, best for your card and trained for; what the name means; Remove.

Each model shows its size, file and context — *now*, *best for your card* and *trained for*. **Remove** asks first and moves the files to the trash, where you can restore them. A model that is only a link loses only the link.

6.5 Context size

The context is how much text the model can keep in mind at once. The app calculates the largest context that fits your graphics card from the model file itself. **Set context** writes it to the local LLM's settings, with a backup and Undo. Agents need a fairly large context; when you connect an agent to an LM Studio model without a context set, the best one is set in the same step.

6.6 On this PC

Lists GGUF files the local LLM does not read yet — in its models folder, the other local LLMs' folders, `~/Downloads` and `~/Models`. **Add** makes one its model without a second copy (a link), except for Ollama, which imports its own copy.

6.7 Moving and deleting models

Right-click a model of llama.cpp, KoboldCpp or LM Studio › **Move to…**, or drag the model box onto another local LLM's folder. First you see how much moves and which agent uses the model. Undo moves the files back. **Delete…** asks first and moves the files to the trash.

6.8 One models folder

Settings › **Models folder** sets one folder for every local LLM's models. **Reorganize models…** previews how the files would be sorted (`<folder>/<publisher>/<model>/`), and nothing changes until you press **Reorganize**. With **Keep old paths working** (on by default), a link stays at every old place. **Undo the last reorganize** puts everything back.

> **CAUTION** Moving models to another disk copies them first and removes the originals only when the copy is complete. Make sure the target disk has enough free space; the app refuses the move if it has not.

CHAPTER 7

Connecting by drawing

Press on a box on the map and drag to another to connect them. Every change shows exactly which lines of which file change before anything is written, keeps a backup, and can be undone from the bar that follows.

7.1 Agent ↔ local LLM

Drag from an agent to a local LLM, or back. Choose the model and whether the agent uses it right away. To change a connection, select its agent (or click the line): every line that can be moved or removed shows an **orange hook** at its end. Drag the hook to another local LLM — or onto empty space to remove it; hovering a hook says what dragging it does. With an agent selected, clicking a model of a connected local LLM makes that model the agent's model.

Supported agents: Hermes, Pi, Oh My Pi, OpenCode, OpenClaw and DeepSeek Harness.

7.2 Agent ↔ MCP server

Drag from an agent to an MCP server. The server's settings are copied into that agent's own format. Supported: Claude Code, Codex, Copilot, Crush, Cursor, Gemini, Goose, Grok, Hermes, Muse, Oh My Pi, OpenClaw, OpenCode and Pi. Settings files that may hold keys are always written so that only you can read them.

7.3 Agent ↔ skill folder

Drag from an agent to a skill folder. Pi and Hermes get the folder in their settings; every other agent gets one link per skill in its own skills folder. A folder an agent reads by itself (most read `~/.agents/skills`) cannot be taken away from just that agent.

7.4 Found on this PC

Every time the window opens (and with **Refresh**), the app looks for MCP servers and skill folders on this computer, without starting anything: programs that offer an MCP server in `~/.local/bin`, `~/Applications`, `/opt` and the folders added on Setup, and skill folders that apps brought along or that are in an added folder. What no agent uses yet is on the map as a faded box without lines. Nothing is connected by itself.

1. Drag from an agent to the faded box.
2. The connect page shows how the server will be started — the command its own help text names, for example with a folder it may read and write — and exactly what changes in the agent's settings.
3. **Connect** writes it. The box is no longer faded, and the app checks right away that the server answers (see When a connection fails).

An app's MCP server runs on its own: the app's window does not have to be open. Programs inside an AppImage that has not been unpacked are not on the map; **Add MCP server** › **On this PC** unpacks them first.

> **TIP** Keep apps with an MCP server and your own skills in one folder each — for example on a disk that several systems share — and add those folders on Setup. On another computer, add the same folders there, and everything in them is on its map too.

7.5 Local connections in the Flutter preview

Open Connections, then Add MCP server. Enter a name, choose stdio, HTTP or SSE, and select the installed agents that should receive it. For stdio, enter the executable separately and put one argument on each line; spaces within a line stay in that argument. Add environment variables or HTTP headers as name/value pairs. Values are obscured. Preview lists recipients and file-change counts, without returning configuration values. Existing definitions with the same name are disclosed before replacement. Apply saves through the existing backup and Undo flow. Maprisma does not start or test the server here; agents may start it when they read the saved configuration.

Add skills selects an existing folder containing skill folders, each with SKILL.md. Choose the parent skill-set folder, then the agents. Preview shows links or adapter settings before Apply. Original files stay in place; links follow later edits, and skills can include scripts agents may run. Native folders already read automatically need no new link. Discovery checks at most 200 entries per directory, one category level deep, with 1,000 entries in total; choose a smaller folder if needed. A preview supports up to 32 agents and 200 links. Undo removes added links or restores settings while preserving later changes. Online catalogs, downloading skills and MCP tool checks are not part of these local forms.

7.6 Found on this PC in the Flutter preview

In the Flutter preview, the app looks at start, in the background and without running anything, for programs on this PC that offer an MCP server (their files hold the protocol's own words; how to start them comes from their own usage text) and for skill sets that apps brought along. Big programs are read once and remembered. Open problems says **New on this PC** for what was not shown yet; **Add MCP server…** or **Add skills…**, or **Found on this PC** in Connections, lists them, the new ones with a ★. **Add…** opens the form filled in with the name and the start command, or the folder, to check and change before the preview. AppImages are never run by the look itself. **Look inside…** unpacks only each one's programs, closed in by bubblewrap (no network, no home folder, only a temporary folder writable), reads them and deletes them again; what was found is remembered. Adding a server that lives in an AppImage first asks to unpack the whole AppImage into `~/Applications/<app>-<version>/` (**Unpack**), the same way. A newer AppImage of an app looked into or unpacked before is only named (**New version found**): a file with a newer name is no proof of an update, so it is looked into like any other.

When an app is found in a newer version while agents, the list of MCP servers or a link in `~/.local/bin` still start the older one, the list shows it with its versions and who uses it. **Update…** (after unpacking the newer AppImage, if it is one) previews one change for all of it: each entry gets the new program (arguments into the old folder move along), the links point into the new folder, and the old folder goes into the backup. It is kept there for 7 days; Undo brings everything back.

7.7 When a connection fails

Right after a connection is written, the app checks whether it will work: whether a local LLM is installed and offers the model, whether an MCP server answers, whether the skill links are intact. A connection that fails stays on the map as a **dotted red line** with a **!**; one with only warnings is **dotted yellow**. Click the line or its **!** for hints:

- **Test again** checks once more; when it passes, the line becomes normal.
- **Try again…** opens the connect page for a connection that was never written.
- **Remove connection** removes it.

CHAPTER 8

MCP servers

MCP servers give agents extra tools. **+ MCP server** on the map or under the cards opens the page for adding one: a list on the left, the chosen server's page on the right.

8.1 Where servers come from

Tab	Contents
Recommended	The MCP project's reference servers (Filesystem, Fetch, Git, Memory, Sequential Thinking, Time, Everything) and official servers of well-known makers (GitHub, Playwright, Chrome DevTools, Brave Search, Context7), grouped by purpose and pinned to their newest version.
MCP registry	A search of the official MCP registry. Each result names its publisher as the registry checked it. The search can take ten seconds or more. The results come 20 at a time; the line under the list pages through them and shows where you are, for example **‹ Previous 20** · 21 – 40 / 200 · **Next 20 ›** (with */ 40+* when the registry does not say how many there are).
On this PC	Programs on this computer that offer an MCP server, found by reading their files — nothing is started. AppImages are only looked into when you click **Look inside…**; they are unpacked closed in by bubblewrap (no network, no home folder), so `bwrap` must be installed.
By hand	A command line, or a web address with headers.

Not sure which server you need? On the **MCP registry** tab, the **Ask an agent** card under the search takes a request in your own words — for example *read and write my Notion pages*. **Ask** (or `Ctrl`+`Enter`) has the agent for AI checks turn it into a few search words; the app searches the registry list it has downloaded with them, as the normal search does, and the agent picks the servers that fit from what was found, each with why. It does not search the web, sees only your request and the servers' public registry texts, and can only pick servers that are really in the list. A spinner and the time taken show while it works; after a few seconds to a minute the picks appear in the result list, up to 10 (**Suggestions** at the right of the agent line: 5, 10, 15 or 20). A search you type afterwards replaces them.

On this PC also looks in the folders added with **Add folder…** on Setup › **Local MCP servers**: in each folder itself and in the app folders in it, such as `photocraft-0.16.0/usr/bin`.

8.2 Setting a server up

The server's page says who made it and how well that is checked, what it does (with things to ask an agent once it has it), how it runs — a program through `npx`, `uvx` or Docker, or a web service — and what it needs, such as a folder or an API key with a link to where you get one. Below it, **Set it up**:

1. Fill in the fields the server needs.
2. Tick the agents that should get it.
3. Click **Test** to ask the server for its tools.
4. Click **Add**: all ticked agents are written at once, with one preview and one Undo.

> **NOTE** Secret values such as API keys are passed in the environment or in headers, never on a command line where other users of the computer could read them. A server that would need a secret on its command line has to be set up by hand.

8.3 MCP servers in the Flutter preview

In the Flutter preview, **Find MCP servers…** in **Connections** opens the tabs **Recommended** (pinned to their newest version when the card opens) and **MCP registry** (20 results at a time with **‹ Previous 20** and **Next 20 ›**; the registry's list is downloaded in the background for a faster search). A server's page says who made it, what it does, what it needs and **How it runs**; the way that can run on this PC is marked **recommended**, and a missing `npx`, `uvx` or Docker is named. Fill in the fields it needs (secret ones are hidden), check the **Name**, tick the agents and click **Preview**: the preview, **Apply** and **Undo** are the ones of a connection change. Only the values you typed go to the service; the command itself comes from the server's entry and is checked there again. A secret that would end up on a command line is refused; such a server has to be set up by hand.

Test starts the chosen way once (or asks its address) and lists the tools it offers, before anything is written. A server already set up shows **Tools** in its card under **Connections**: the answer it gave last time, with the date, and **Ask the server** (or **Ask again**), which starts it once the way its first agent does and stops it again. Its **Prompts** are listed too. The answer is kept for the next time; keys in its settings go to the asking process privately and are never shown.

8.4 Managing servers

ⓘ **Details** on an MCP server card shows each agent's settings file, the command or address, and on request the server's tools; values of keys are never shown. **Remove** takes the server from every agent. Servers stay in the list when no agent uses them any more, so a line can connect them again.

When a newer version of an app that provides an MCP server turns up in `~/Applications`, a note names it. **Update…** unpacks the new version, points every agent at it and keeps the old one as a backup for seven days. **Check for updates** (in **On this PC**) does it for every such app at once after your click: it looks inside the new versions, offers all updates in one preview with one backup and one Undo, and then tests each updated server; a note names any that does not answer.

CHAPTER 9

Skills

A skill is a folder with instructions that teach an agent a particular job. **+ Skills** opens a page laid out like the MCP page.

9.1 Finding skills

- **On this PC**: skill folders that apps brought along and no agent reads yet.
- **From the web**: a search of the skills.sh directory, another site with the same API, or a GitHub repository with skills. A skill is fetched from its GitHub repository into a folder of your choice, by default `~/.agents/skills`.

> **CAUTION** Skills can contain scripts that an agent may run later. The page shows the repository and lists the scripts before anything is fetched — check them.

On this PC also looks in the folders added with **Add folder...** on Setup › **Skills**.

9.2 Skills from the web in the Flutter preview

In the Flutter preview, **Find skills on the web...** in **Connections** searches skills.sh. A result names its GitHub repository and how often it was installed; choosing it fetches the skill's files — at most 150 files and 30 MB, no links, nothing outside its folder — and runs none of them. The card shows its description, how many files it has and which scripts an agent could run later. Choose the folder it goes **Into** (the shared `~/.agents/skills` or one of your own skill folders), tick the agents under **Also for** and click **Preview**: the new files are listed, and **Apply** writes them with a backup, so **Undo** removes them again. A skill whose folder is there already is not overwritten. The fetched files wait in a private folder until then; each is checked against what the preview showed before it is written.

9.3 Skill folders

Each skill folder card shows its skills and the agents that read it. A folder that is shared through links shows up once, where it really lives. ⓘ **Details** lists every skill with its description and how each agent gets the folder.

CHAPTER 10

Settings and Help

10.1 Flutter themes and map lights

In the Flutter app, Settings offers System, Light and Dark with small previews. System follows the desktop appearance and the existing desktop palette; Light and Dark set a fixed brightness with matching backgrounds and readable colours. The choice is saved and also applies to task cards. The GTK appearance preference is kept separately. With System, a new desktop theme shows within two seconds, also while the app is busy with a download, an install or a change.

Map light frame rate offers 20 fps (default), 30 and 60. It is a maximum for flowing map lights, not the automatic scan interval. The light speed stays the same; higher rates add intermediate frames. Actual delivery depends on the display and available rendering time. Lights off, reduced motion, an unfocused window or a hidden map still stop the animation. The choice is saved.

Settings also offers **Start local LLMs with their agents**: when an agent connected to a local LLM starts, its LLM starts too, and when the last such agent closes, the LLM stops and its GPU memory is free again (a user service does this, also with the window closed). **Agent for AI checks** chooses which installed agent is offered first for AI help and the Smart Check on the System Map; its tokens are used for them. It lists the agents the System Map found able to do that.

10.2 The popup under the bar on Omarchy

On Omarchy the Flutter app opens as a popup: a floating window at the top right under the bar, without a title bar, like the plugins' popups. `Esc` on the main page or a click elsewhere closes it; starting the app again (or the bar icon, which runs `maprisma --toggle`) brings it back where it was, with everything as you left it. A file chooser the app opens does not close it. Settings › **Window** chooses **Automatic: a popup under the bar on Omarchy** (the default), **A popup under the bar** or **A normal window**; a change applies the next time the window opens. On other desktops the app is a normal window.

10.3 Usage figures and corners from Omarchy in the Flutter preview

When an agent's details open in the Flutter preview and Omarchy collects that agent's usage (plan, limits and when they reset), the app asks Omarchy for fresh figures in the background if the last ones are older than ten minutes or older than the agent's last sign-in, then shows them; at most once every ten minutes per agent. Settings › **Corners** sets the corners of the window's cards, dialogs and the map's boxes: **Like windows** follows Hyprland's window rounding (with the number of pixels shown), **Rounded** or **Square**.

10.4 Settings

The gear (`Ctrl`+`,`) opens the settings. `Backspace` or `Esc` goes back.

Setting	What it does
Language	The system language, or English, Deutsch, Français, Español, Italiano, Português, Nederlands, Polski.
Look OMARCHY	The Omarchy theme colours, or GNOME's Adwaita look with the system's light/dark mode and accent colour.
Map animations	Light runs along the lines on the map: **Agents and other programs** (a working agent's line, and a local LLM's line to a model another program uses), **Only my agents**, or **Off**. A small animated map under the setting shows what each choice moves.
Usage statistics box	The numbers along a working agent's line (see Working with the map): on or off, which of **Tokens per second**, **Context size** and **Context used** it shows, and its **Text size** (7 pt by default).
Subagents	For the Subagents page: how long finished subagents stay (**30 minutes** by default; 10, 30, 60 or 120) and how many at most (**10** by default; 5, 10, 20 or 40). Working ones are always shown.
Costs	**Currency**: US dollar, or Euro at the European Central Bank's daily reference rate (fetched once a day from ecb.europa.eu). **A day's limit** for an agent paid per token (**100** by default; 10 to 500). **Yellow from** and **Red from**: the share used at which a cost label or a battery turns yellow (75 % by default) and red (90 %). **Notify when red** (on by default): a desktop notification when an agent turns red — once for each limit until it resets, or once a day for a cost.
Terminal tabs above the map	One tab per agent terminal in the window.
Corners	Like windows, rounded or square.
Start local LLMs with their agents	The background watcher described in Starting and stopping with agents.
In the app launcher	The launcher entry; off removes it.
Agents start in	The folder agents start in.
AI features: **Look up new agents**	Ask an installed agent where an unknown agent keeps its settings (on by default).
AI features: **Daily AI checks**	Once a day, an agent looks on the web for new versions of your MCP servers (off by default; it uses the agent's tokens). See Smart Check.
AI features: **Agent for AI checks**	Claude Code or Codex: does the AI checks and look-ups and takes the **Do it with …** and **Fix it with …** tasks.
AI features: **Local helper**	An agent with a model of your local LLM for the AI help that needs no web (off until you choose one). See The local helper.

Setting	What it does
Models folder	One folder for all models, with **Reorganize models…**.
Add-ons MAPRISMA	**Look for add-ons online**, the Add-ons page, and **Setup**.
About	The version, the author and the license; **Website** opens the project on GitHub and **Report an Issue** its issue list.

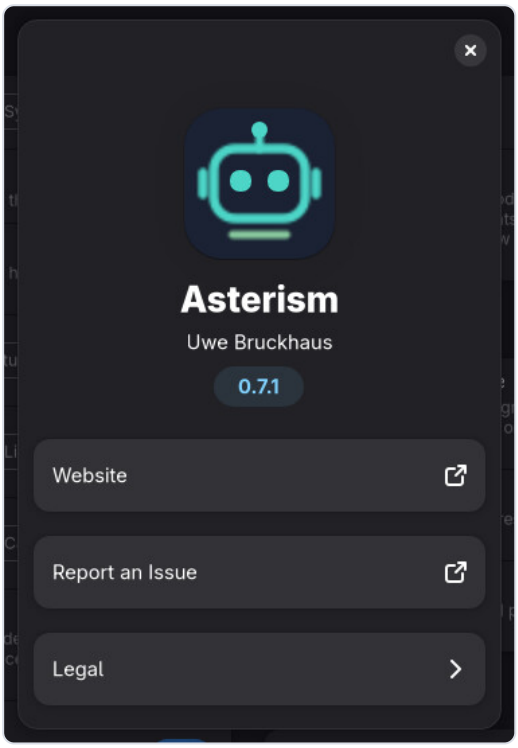


Figure 11 · Settings › About.

10.5 Red limits in the Flutter preview

In the Flutter preview, the app looks once a minute at the plan limits of your installed agents (a 5-hour session, a week…). When one is used up to the red level of Settings › Costs, a desktop notification says so, with the share used and when the limit resets: once for each limit until it resets, also after a restart. **Notify when red** in Settings turns it off. A cost of an agent paid per token is not shown yet in the preview, so neither is its notification.

10.6 Recent changes and Undo after a restart

In the Flutter preview, Settings ends with **Recent changes**: the last 20 changes this app wrote to agents' settings (connections, MCP servers, skills, security settings, app updates), newest first, with the files they touched. **Undo** puts back what was not changed again since — also after the app was closed or crashed, when the **Undo** of the last change is gone. A change whose files were edited again since shows no **Undo**. A change that stopped half way (the app closed while writing) puts back only the files it had already written. The backups stay where they are; Undo only reads them.

10.7 Help

? (F1) opens Help: **Open user manual** opens this manual (the copy that came with your version) in your PDF viewer; **Disk space** (below); an animation of setting up by drawing; and LLM Monitor, a companion bar plugin that shows what a local LLM is doing (a small animation of its bar item and panel). Both animations are drawn in your theme's colours and follow a theme change at once. The ⓘ legend in the header explains every dot, line and mark of the map.

The filled **i** in the header opens **Information**: cards for the author's other plugins, read from the Omarchy Plugin Marketplace when the page opens, at most once an hour (without the network, the list this version knows). Clicking a card opens the plugin's page in the marketplace.

10.8 Help in the Flutter preview

In the Flutter preview, **?** in the header (or F1) opens **Help**: **Open user manual** opens this manual in your language (the copy that came with the app, else the English one) in the desktop's PDF viewer, and **Where to start** leads to **Your first agent**, **Find existing AI**, **Providers and models** and **Connections**.

10.9 Disk space

Disk space: agents, local LLMs and models shows how much room your AI setup takes and where it is. It is measured in the background each time Help opens (again after ten minutes, or with **Measure again**), and only read — nothing is deleted from here. Its heading always shows the totals in one line (*Agents · Local LLMs · Models · In all*); click it to fold the section away. **Disks**, **Agents**, **Local LLMs** and **Models folders** inside start folded, each showing its sum and largest entries; click one to open it.

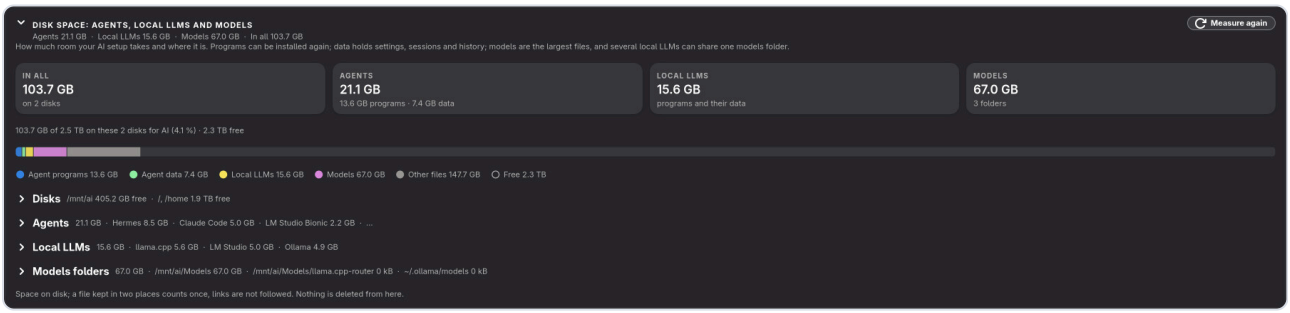


Figure 12 · The totals, the AI share of the whole disk space, and the folded sections with their sums.

- **Tiles**: everything together, then agent programs, agent data (settings, sessions, history), local LLMs (programs and their own data) and models.
- **Bar**: the whole space of the disks the AI setup is on. The coloured parts are the AI setup, grey is everything else on those disks, and the empty rest is free; the line above it gives the AI share in percent. Hover over the bar for exact sizes.
- **Disks**: each disk with what the AI setup takes on it and how much is free.
- **Agents**, **Local LLMs**, **Models folders**: one row each, largest first. Open a row to see every folder with its size and number of files, and open it in the file manager. A program folder says how many versions are kept; for an agent with several old versions in mise, a tip names `mise prune`, which removes the unused ones. A models folder says which local LLMs use it, or that it holds only links to models in another folder.

> **TIP** Programs can be installed again and models downloaded again; the agents' data holds your sessions and settings. When space runs out, look at old program versions and at models you no longer use first.

Sizes are space on disk: a file kept in two places counts once, and links are not followed.

CHAPTER 11

Smart Check

Smart Check lets an installed online agent — Claude Code or Codex — look at this PC's AI setup, find updates for your MCP servers and do fixes for you, and shows what that cost in tokens. A local helper — an agent with a model of your own local LLM — can take the questions that need no web, on this PC and without tokens. Every run shows its progress in a box over the window and is kept in the Protocol. The two sparkles in the header open it; they show only while one of these agents is installed. Settings › **Agent for AI checks** picks which one.

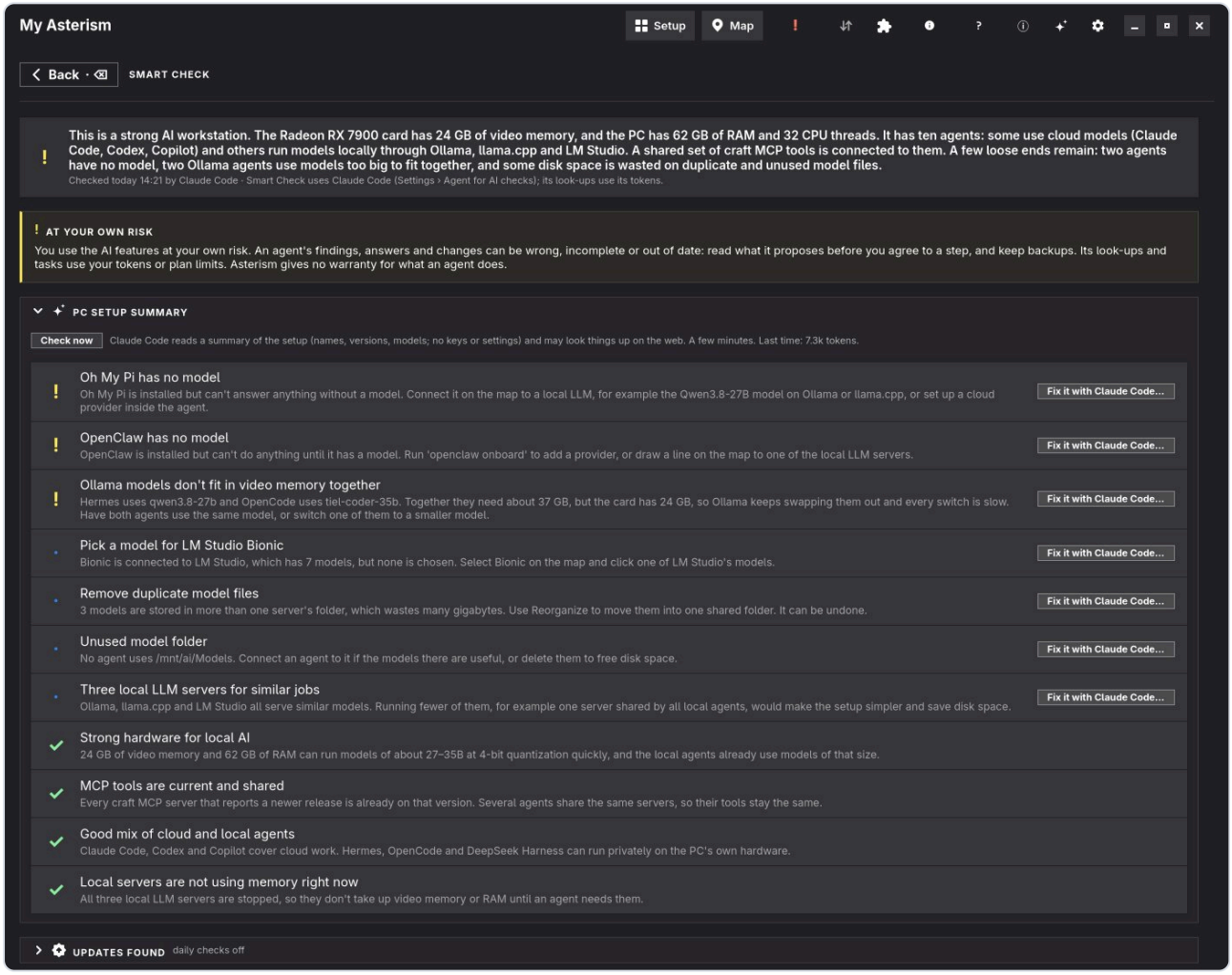


Figure 13 · Smart Check after a check by Claude Code: problems (!), tips (·) and what is good (✓).

> **AT YOUR OWN RISK** An agent's findings, answers and changes can be wrong, incomplete or out of date. Read what it proposes before you agree to a step, and keep backups. Its look-ups and tasks use your tokens or plan limits. The app gives no warranty for what an agent does. The same note heads Settings › **AI features**.

11.1 PC setup summary

Check now gives the agent a short text about the setup — the graphics card and memory, the agents with their models, the local LLMs, the MCP servers with their versions, the skills and what Open problems found — and lets it look things up on the web. No keys, headers or settings values are in it. It takes a minute or more in the background; the line under the button says how many tokens the last check used. The answer is a short summary and a list of findings: **!** problems, • tips and ✓ what is good. Answers that do not hold up are left out.

Fix it with … next to a problem or tip opens the agent in a terminal tab in the window with that finding as its task: it looks whether it is so, explains, and asks you before each step that changes anything. The app starts it so that it must ask — Codex in a read-only sandbox that asks for more, Claude Code with every command and edit set to ask, whatever its own allow-list says — and without its MCP servers. The finding is handed over as data to check, not as instructions. Moving MCP servers to a new version is left to the app's **Update** (below), which changes every agent's settings at once, with a backup and Undo.

11.2 Updates found

New versions of your MCP servers, from two sources:

- **The daily AI checks** (Settings › **Daily AI checks**, off by default): once a day the agent looks on the official release pages for newer versions of servers whose version the app can tell — apps unpacked in `~/Applications/<app>-<version>/` or as an unpacked AppImage in any other folder (the program in `<folder>/<app>-<version>/usr/bin/`) and npm or PyPI packages pinned to a version. **Check now** on this card asks at once, also while the daily checks are off. Each find has **Release page** and **Do it with …**: the agent downloads the new AppImage in a terminal tab, asking before each step.
- **The app's own scan**: a newer version is already on this PC, but an agent still starts the old one (the same as the update on the server's card on Setup).

Once the new version is on this PC, its line says so and offers **Use it…**: the update preview shows which agents move to it; **Update** changes them all, keeps the old version with the backup and tests each server afterwards. A server whose agents all start the new version drops off the list.

> **TIP** After an action started here — an update, or a task in a terminal tab — **Back** brings you to Smart Check again, up to date.

11.3 While an agent works

Every AI help run you start — Check now, Ask an agent on the Models and Add MCP server pages, Check now for updates, Ask on a local LLM's Performance card — shows a box in the middle of the window: who is asked (the agent and its model), its steps (a spinner that turns into ✓, or ✕ when one failed), the time so far and, at the end, the tokens it used.

- **In the background** hides the box; the run goes on, and a message says when it is done.
- **Cancel** ends the agent at once (its program and everything it started).

- When it fails, the box stays with what went wrong until you close it; the text can be selected and copied.

Look-ups on the web (model advice, MCP updates) run with the web and nothing else: without the agent's MCP servers, connectors, plugins and hooks.

11.4 The local helper

An agent connected to a local LLM can answer AI help with one of that LLM's models: on this PC, with no web, no tokens of a plan and nothing leaving the computer. It answers what needs no web: Smart Check's summary, which MCP server fits, and the tips on a local LLM's Performance card. Model advice and MCP server updates still go to the online agent, since they need the web.

The app suggests one — the largest chat model your graphics card holds (speech, embedding and tokenizer models are left out) — on Smart Check, with **Use it**. Settings › **AI features** › **Local helper** lists every agent and model that can be it, the suggested one first, and **Off**. Nothing is used until you choose it. Before it answers, the app starts the local LLM and loads the model (a step in the progress box). The helper runs with no tools, no MCP servers, no extensions and no project files. Any agent whose add-on says how to ask it one question offline can be the helper; Pi is the first.

11.5 AI checks and the local helper in the Flutter preview

In the Flutter preview, Settings › **AI features** has **Daily AI checks** (off by default) and, below it, the result of the last check with **Check now**, which asks at once, also while the daily checks are off. The check runs in the background when the window opens and is looked at every hour; the agent for AI checks is asked at most once a day (after a failed look-up, again after six hours). What it finds is listed in Open problems under **Updates and suggestions**: **Release page** opens the release in the browser, **Do it with …** first shows the task as it will be sent and starts it in a terminal tab only after **Continue**, and **Use it…** opens what was found on this PC when the newer version is already here. Checked are only servers whose version the app can tell (see Updates found); other servers are not sent.

The local helper is chosen in Settings › **AI features** › **Local helper**; the suggestion for your graphics card is marked **(suggested)**. Once chosen, it is offered first under **Ask** on a local LLM's Performance card.

11.6 Protocol

Every AI help run is kept, so that you — or someone who knows the setup well — can look over what was done. Each entry shows when it ran, the agent and model, the result (worked, failed, cancelled), the time and the tokens. Opened, it shows the whole text: the options that closed the agent in, the steps, each question exactly as it was sent with the agent's answer, what the app changed from the answer (with the backup, for Undo by hand) and, for a task in a terminal tab, what the terminal showed when it ended. **Copy** puts an entry on the clipboard for a report or a second opinion; **Open the protocol folder** shows the files.

The protocol keeps the last 200 runs in `~/.local/state/ai-agents-map/ai-help/`, readable only by you. Questions hold names, numbers and settings, never keys; a terminal's text holds what was on the screen.

11.7 Token statistics

Tokens used today and in the last 30 days, a chart per day, and the totals by agent and by plugin (other Omarchy plugins that use the same AI helper show up here too). **API value** is what the runs would cost at API prices, and the line under it says whether your subscription covers it or it is charged to an API key. A task in a terminal tab counts in its agent's own session; the agent's details page shows those tokens.

Below, one card per kind of task — the setup check, MCP server updates, looking up new agents, fixes — with its runs, tokens and the latest results.

CHAPTER 12

Group cards and add-ons on the map

12.1 Expand a category

In the Flutter preview, every AI Map column has one group card. A dashed card means nothing is set up; its found count reports local detection. These placeholders are display elements, never connected nodes or AI input. Click Expand, double-click the card or press Enter while it has focus. The group appears alone in the centre. Back, Esc, Backspace or the surrounding background restores the map's zoom and selection. The System Map uses the same controls for hardware, disks, programs, tools and folders.

12.2 Manage add-ons

Focus groups show On, Off, Found on this PC and Not installed as applicable. Use the overflow menu or right-click for switching, Add-on details, a newer signed version or provider sign-in through an installed agent. Details disclose reads, sends and capabilities. Switches apply immediately with one Undo snackbar. Changes are blocked while this app owns running agent sessions or an installation is active. Agents in other terminals do not block add-on management. Look for add-ons online explicitly loads the signed catalog; merely expanding never requests it. Review Install or Update before execution. New add-ons stay off. Removing a user add-on makes a backup and preserves programs and data; Undo is available until the app closes. Bundled add-ons stay with their package.

After a confirmed removal, old backups become eligible for cleanup after 30 days. Backups with an active Undo are kept; links, unrelated folders and foreign-owned folders are skipped. Undo lasts until this app closes. During first use, Welcome's Agents button reopens the same Agents focus with its fixed Switch on / Later footer; it does not finish welcome or open a plain group view.

Install from file… takes an add-on .zip you downloaded yourself. Its own signature decides, as for a catalog download: the file is checked first and shown like a catalog entry (name, version, kind, what it reads and sends, and whether it replaces an installed version), and nothing is installed until you confirm. The file is read again then and must still be the one you reviewed. Only agent and provider add-ons can be used in the Flutter preview so far.

12.3 First use and existing flows

Welcome appears above the map, with local Look at this PC / Skip and a System Map link. Mount existing data disks before searching. The Agents group opens with detected agents marked. Its fixed Switch on / Later footer stays visible while the list scrolls. Switch on is one batch with one Undo; Later preserves choices. Other enabled add-on kinds are preserved. Later starts do not auto-expand. Models links to existing download/import controls and model-folder reuse; MCP servers and skills links to Connections. New MCP definitions and new skill-folder installation remain unported. The rocket button retains the first-agent guide. Setup and Add-ons pages have been replaced by these groups.

CHAPTER 13

Editing the Flutter AI Map

Drag an agent box to a provider, MCP server or skill set, or draw in the opposite direction. Click an agent and then a model of a connected provider to choose that model. Drawing a provider to its own model opens an agent selection; local providers that support loading can also load without choosing an agent. Every configuration edit opens Preview, then Apply, progress and a result with Undo. Cancel writes nothing.

Select a box to reveal orange connection hooks. Drag a hook to another compatible provider or model, or to empty space to disconnect. A provider’s model hook switches all configured users together, or disconnects them and unloads where supported. Busy runtimes refuse unloading. Online provider lines use the agent’s registered sign-in or sign-out command in a reviewed terminal.

Hover highlights neighbours. Shift-click adds boxes, with at most one agent selected. Pin keeps a box highlighted. Fold a card with its arrow, drag its header to move it, or use the card menu. Drag a box’s reorder handle or choose Move up / Move down. Layout and pins are saved separately from connections; task cards retain selection and zoom.

Keyboard users can open Actions on a box, choose Connect or Move connection, and activate the destination box. Disconnect removes the pending connection; Cancel or Esc cancels. Card arrows and movement menus, Pin and reorder menu items provide alternatives to dragging. Open shows existing details and management actions. Agent buttons start in the saved default folder, show or stop an owned console, or offer a reviewed external instance to stop.

All lights, Agent lights and Lights off control estimated activity. Lights stop when no line is active, the map is hidden, the window loses focus, or reduced motion is enabled. Failed lines open bounded hints with Repair or Disconnect. Undo restores configuration and failed-link metadata; runtime load/unload and sign-in/sign-out actions are not reversed. A partial model operation keeps the configuration backup and offers Undo.

CHAPTER 14

Task cards on the map

Connections, agent actions, model removal and add-on changes open a task card in the Flutter preview. When the map area is at least 900 logical pixels wide and the task has known map items, a read-only map appears to the left of the card. It shows those items and their direct neighbours, with lines leading to the card. Empty columns are omitted. The main map keeps its zoom, position and selection. The provider overview uses the full width; individual runtime actions show a half card when their map items are known.

Review the proposed changes before confirming. The same card shows progress and the result; use Undo where offered. Back or Esc cancels a review or closes a finished card. Running work cannot be closed with Back or Esc; only operations that support cancellation show Cancel. Model search/downloads, the add-on catalog and the first-agent guide use full cards. Narrow windows and tasks without known map items also use full cards. Settings, terminals, usage and subagent details retain their existing views.

CHAPTER 15

System Map

15.1 First use and separate mounts

Flutter development preview: switch between AI Map and System Map. Each keeps its zoom and selection. Mount existing data disks before choosing Look at this PC. The bounded search reads local metadata, never runs detected programs and sends nothing. Skip keeps disk search off. Hardware, disks, agents, runtimes, supporting tools and folders appear together. Missing disks remain visible.

15.2 Saved map and refresh

The private saved map opens immediately. Startup and visits refresh status without repeating the full search or size measurements. Refresh explicitly searches mounted disks and measures folder sizes. After the first search, newly mounted disks are searched once. While this page is visible, mount checks run no faster than every 15 seconds and respect the background interval, Manual-only and focus pause settings. Tools may need an explicitly confirmed version check.

For a disk that is permanently gone, select its grey node and Forget this disk. Confirmation removes only its saved map entries, not files or settings. Configured folders can appear again on refresh. Runtime discoveries show the launch path (including current) and the resolved build separately. Sizes follow known wrapper targets; unattributable program sizes are hidden. With an explicit settings profile, system-map.json is kept beside settings.json. The default map uses the application cache folder. New durable Undo journals always live in model-moves/ beside settings.json; existing journals in the old default cache location remain readable. Keep legacy journals until their moves are undone.

15.3 Reconnect, move and undo

Select a found component for Check, Reconnect or Use folder. Hold and drag a runtime onto a model folder to preview Reconnect; drag a folder to the AI Map runtime to preview its configuration. Confirmed connections create configuration and backups, without starting services. Drag a GGUF folder onto a disk for a bounded move preview. Running readers block moves; cross-disk moves copy and verify first. Supported links and references are updated. Undo preserves new files. Custom wrappers and unsupported stores require a reviewed manual or AI plan. Mount suggestions are text only: fstab is never edited.

15.4 Analysis and AI help

Hardware health, GPU stack guidance and findings appear on nodes. Check this system with AI requires consent to the exact summary and prompt: the chosen agent and any online model service receive folder paths and may consume tokens. Fix it with AI and Simplify your AI setup open guarded tasks for review. No credentials are included. Usage on the AI Map shows recent runs and token totals. Review duplicates and unused services before consolidating; keep backups.

CHAPTER 16

Healthy & Security check

A coding agent reads and writes files and runs commands on your computer. How much it may do *without asking you* is set in the agent's own settings, with different words in every agent. The **Healthy & Security check** brings this together: on the details page of every agent and of every local LLM, four cards show whether it works, what it may do, what that means in plain words, and what lowers the risk.

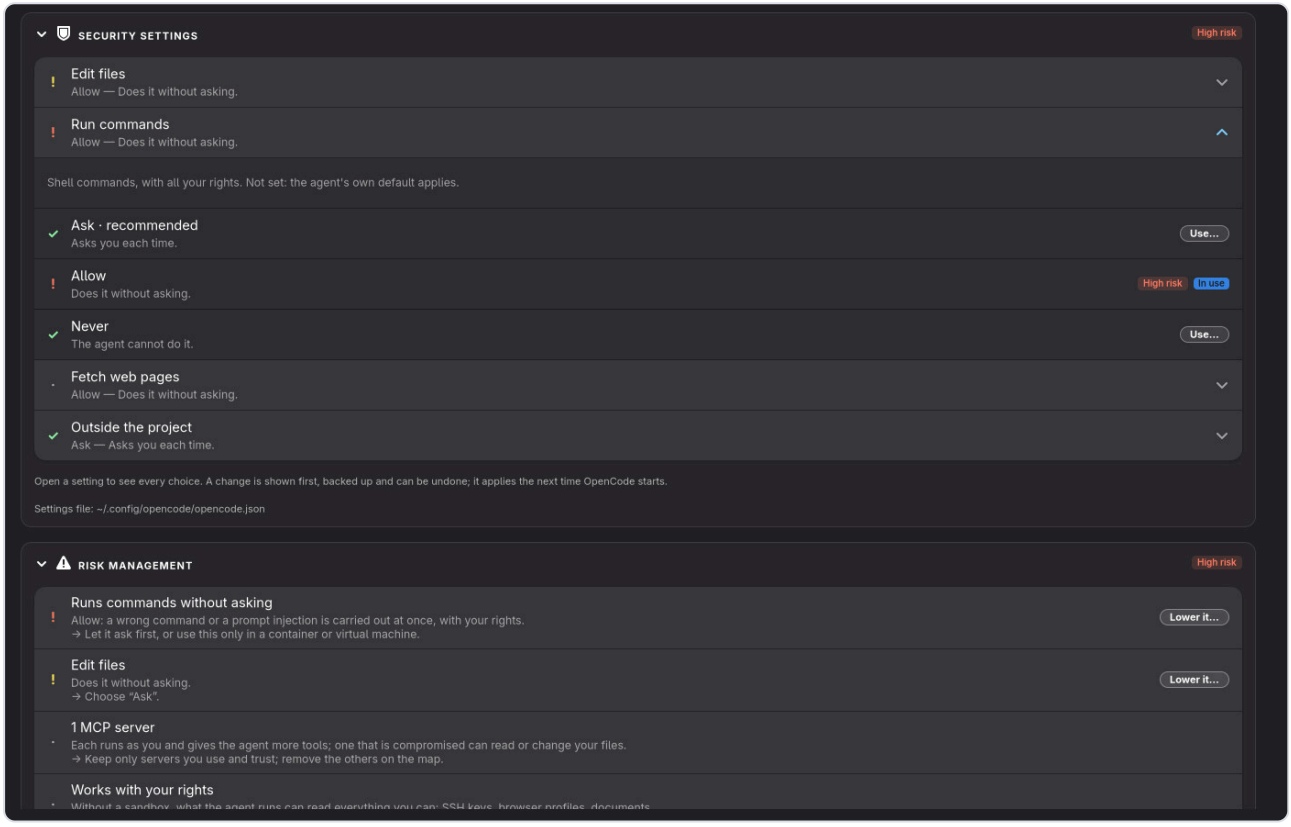


Figure 14 · OpenCode's Security settings, with **Run commands** opened, and the start of its Risk management.

16.1 Reading the cards

The badges on the folded cards sum it up: **Healthy**, **Needs a look** or **Not set up** on Health, and **Low risk**, **Medium risk** or **High risk** on Security settings and Risk management. Open a card for its details:

Card	What it shows
Health	Every setup check with a ✓, or what to do next: installed, signed in, its model and local LLM, MCP servers and skills.
Security settings	The agent's settings that decide what it may do on its own, each with its value now and one sentence on what that means. The mark in front tells the risk: ✓ safe, · low, a yellow **!** medium, a red **!** high.

Card	What it shows
Risk management	What the settings mean together, most serious first, each with why it matters and → what lowers it. Protections that are in place are listed with a ✓.
What the words mean	Sandbox, approval, allow and deny rules and the other words the cards use, explained (see The words explained).

Allow and deny rules and trusted folders the agent has (for example Claude Code's `permissions.allow` list) are listed below the settings, one rule per line.

16.2 Changing a setting

1. Click a setting to open it. You see what it controls and every choice, each with its risk; the one in use is marked **In use**, the safer default **recommended**.
2. Click **Use…** beside a choice, or **Lower it…** beside a risk. A page shows the change in words (old → new), the risk before and after, and **What changes**: the lines of the agent's settings file.
3. Click **Change**. The file is backed up and written; a message offers **Undo** for 30 seconds. The agent uses the new setting the next time it starts.

> **NOTE** Only the one key changes; comments and everything else in the file stay as they are. A setting that needs more than a line in a file — such as Hermes running its commands in Docker — is shown and explained, with where to change it, but not changed from here.

16.3 Security in the Flutter preview

In the Flutter preview, an installed agent’s details have **Security settings**. The card shows the risk badge, every setting with its choices in a list (a coloured dot gives each choice’s risk, the safer default says **recommended**; a setting the file does not name says that the agent’s own default applies), the allow and deny rules, the settings file, and **Risk management** with **Lower it…** where one setting lowers a risk. Choosing another value or **Lower it…** opens the **Preview** a connection change uses: the lines of the settings file that change. **Apply** backs the file up and writes it, with **Undo** afterwards; only the one key changes, comments and the rest of the file stay as written.

16.4 Health in the Flutter preview

In the Flutter preview, an installed agent’s details and each local LLM in **Providers and models** have a **Health** card: **Healthy**, **Needs a look** or **Not set up**, one sentence, and every check — installed, signed in (only whether a credential file or key is there is looked at), model, its local LLMs, failed connections, MCP servers and skills; for a local LLM whether it runs, its models and the agents connected to it. A check that is not ok says what to do next, with a command to **Copy** where there is one. It opens by itself when something needs doing.

16.5 What is checked for each agent

Agent	Settings shown and changeable	File
Claude Code	Permission mode (Ask first, Plan only, Only what is allowed, Edit without asking, Automatic review, Bypass all checks), sandbox for commands, block bypass mode; allow / ask / deny rules	`~/.claude/settings.json`
Codex	When Codex asks, sandbox (read only, project folder, none), network in the sandbox; trusted folders	`~/.codex/config.toml`
GitHub Copilot	Permission mode (Ask first, Automatic review, Allow everything); trusted folders, sites without asking	`~/.copilot/settings.json`
Cursor CLI	Approval, sandbox, network in the sandbox, web search without asking; allow / deny rules	`~/.config/cursor/cli-config.json`
Hermes	Dangerous commands (always ask, automatic review, never ask), hide secrets; where commands run (shown only)	`~/.hermes/config.yaml`
OpenCode	Ask, allow or never for editing files, running commands, fetching web pages and working outside the project	`~/.config/opencode/opencode.json`
Pi	Its tools: read, write and run commands, or read only. Pi never asks by design, so its tools are its permissions.	`~/.pi/agent/settings.json`

For other agents the card shows their health and their MCP servers, and says where to look in the agent's own settings. For a **local LLM** it shows who can reach it: only this PC (`127.0.0.1` , as it should be) or every device on your network.

16.6 Managing the risk

The risk level is the most serious point the card finds:

- **High risk** — it can do serious things without asking: run any command, or work with no sandbox and no questions. Fine in a throwaway container or virtual machine; on your own PC, let it ask first.
- **Medium risk** — some things happen without asking, such as editing files in the project, or an automatic review decides for you.
- **Low risk** — it asks before anything that matters, or cannot do it at all.

Good habits, whatever the settings:

- **Ask first for commands** on your own computer; allow single, harmless commands (such as `git status`) with an allow rule instead of allowing the whole shell.
- **Switch the sandbox on** where the agent has one (Claude Code, Codex, Cursor CLI), and leave the network in it off unless a task needs it.
- **Start agents in the project folder**, not in your home folder, and keep keys and passwords out of it.

- **Keep MCP servers few**: only the ones you use and trust. Each runs with your rights.
- **Run unattended work** (bypass mode, “YOLO”, Pi with all tools) in a container or virtual machine.
- **Local LLMs on** `127.0.0.1`: their servers have no password.

> **CAUTION** An automatic review (Claude Code's *auto* mode, Copilot's *assisted*, Cursor's *auto-review*, Hermes's *smart*) is a second AI judging each step. It saves questions, but it is a judgement, not a wall.

16.7 The words explained

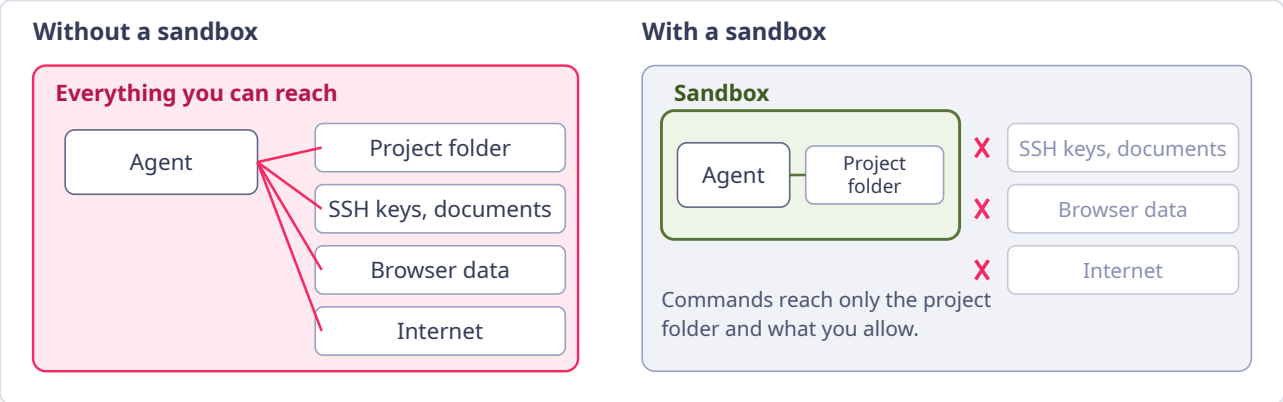


Figure 15 · What a sandbox changes: commands reach only the project folder and what you allow.

Sandbox	A fenced-off area the agent's commands run in. Inside it a command can touch only the project folder (and what you allow) — not the rest of your home folder, your SSH keys or browser data, and often not the internet. Without one, a command the agent runs can do anything you can.
Approval (asking first)	Before the agent edits a file or runs a command, it shows what it wants to do and waits for Yes or No. The less it may do without asking, the more control you keep, and the more often you are asked.
Allow and deny rules	Lists of commands or tools the agent may use without asking (allow) or never (deny). Deny wins over allow. A rule that allows every shell command switches approval off for the shell.
Trusted folder	A folder you said the agent may work in. There it also uses the folder's own instructions and settings, which can come from someone else when you clone a project.
Network access	Whether commands may reach the internet. Code that reads your files and can send data out could leak them; installing packages needs the network too.
Bypass (“YOLO”)	Everything without asking and without limits. Only for a throwaway virtual machine or container.
Automatic review	A second AI model lets harmless actions through and asks about the rest.
Prompt injection	Instructions hidden in a web page, file or tool answer that try to make the agent do something you did not ask for. Asking first, a sandbox and no network stop it.
Secret redaction	Text that looks like a password, token or key is masked before the model sees it or it reaches a log.
Tools	What the agent can do at all: read, write, run commands, search the web. A tool it does not have cannot be misused.
MCP server	A small program that gives the agent extra tools. It runs as you, with your rights.
127.0.0.1 (localhost)	An address only programs on this PC can reach. A server on `0.0.0.0` can be reached from every device on your network.

CHAPTER 17

Add-ons

Everything Maprisma understands about an agent or an LLM provider is an **adapter**: how it is found and installed, its sessions, security settings, MCP servers and skills. In Maprisma every adapter is an **add-on**: the packages bring all of them, signed and switched off, and you switch on the ones you use. Add-ons also bring new agents, new providers, and features beyond the numbers, such as **Insights** (session topics and a summary by Claude). The AI Agents Map plugin has the adapters built in and no add-ons.

17.1 Setting up Maprisma

A new Maprisma opens on its **welcome page**. Every agent and local LLM Maprisma knows is a card with its own icon, in three groups: **AI agents**, **Local LLMs** and **Online providers** (the names of Anthropic, OpenAI and the others on the map). Each card says whether the program is here:

- **Installed** (green) — found the same way the map finds it; the card is already ticked;
- **Used before** — not installed now, but its settings are still here;
- **Not installed** — tick it to install it afterwards from its card.

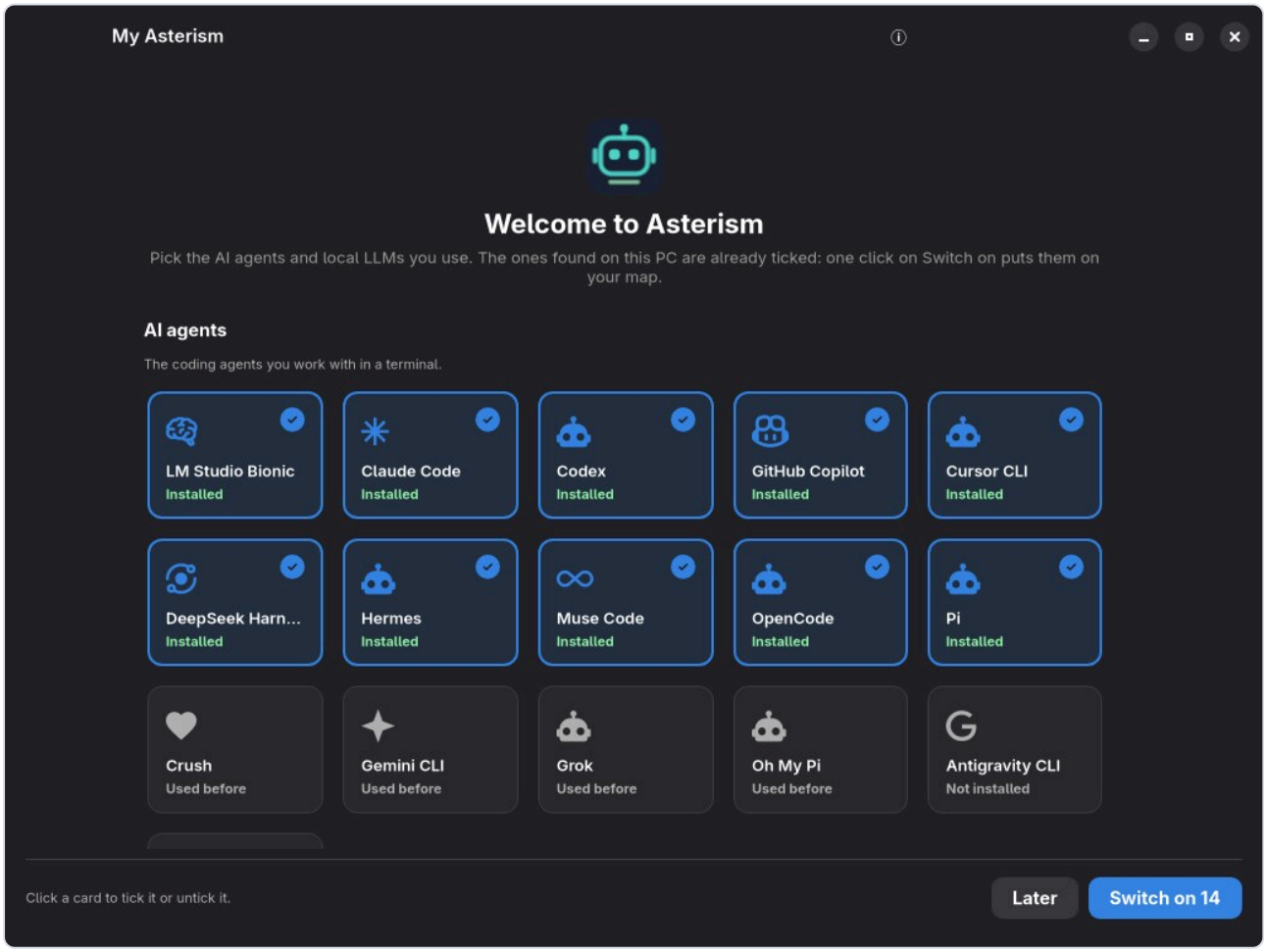


Figure 16 · The welcome page: found programs come first and are ticked.

Click a card (or press `Space`) on it) to tick or untick it; a ticked card has a blue frame and a check mark. **Switch on** (with the number of ticked cards) loads them and shows the map; when you ticked programs that are not installed, **Setup** opens: ⋯ on their cards › **Install…**. **Later** starts with an empty map. As long as nothing is switched on, Maprisma opens on the welcome page again at every start. Afterwards the same cards are on **Setup** (the header button), where agents and providers are switched on and off at any time.

17.2 The Add-ons page

Open it with the puzzle button in the header, from Settings › **Add-ons**, or from any "add-on available" hint. It shows, as cards:

- **Suggested for this PC** — an add-on whose agent or provider is installed here but not shown yet;
- **Installed add-ons** — on, off, or *not loaded* with the reason (for example a changed file);
- **Available** — the other add-ons of the catalog;
- **Built into Maprisma** — agents and providers that no installed add-on brings (when you run Maprisma from its source folder without add-ons).

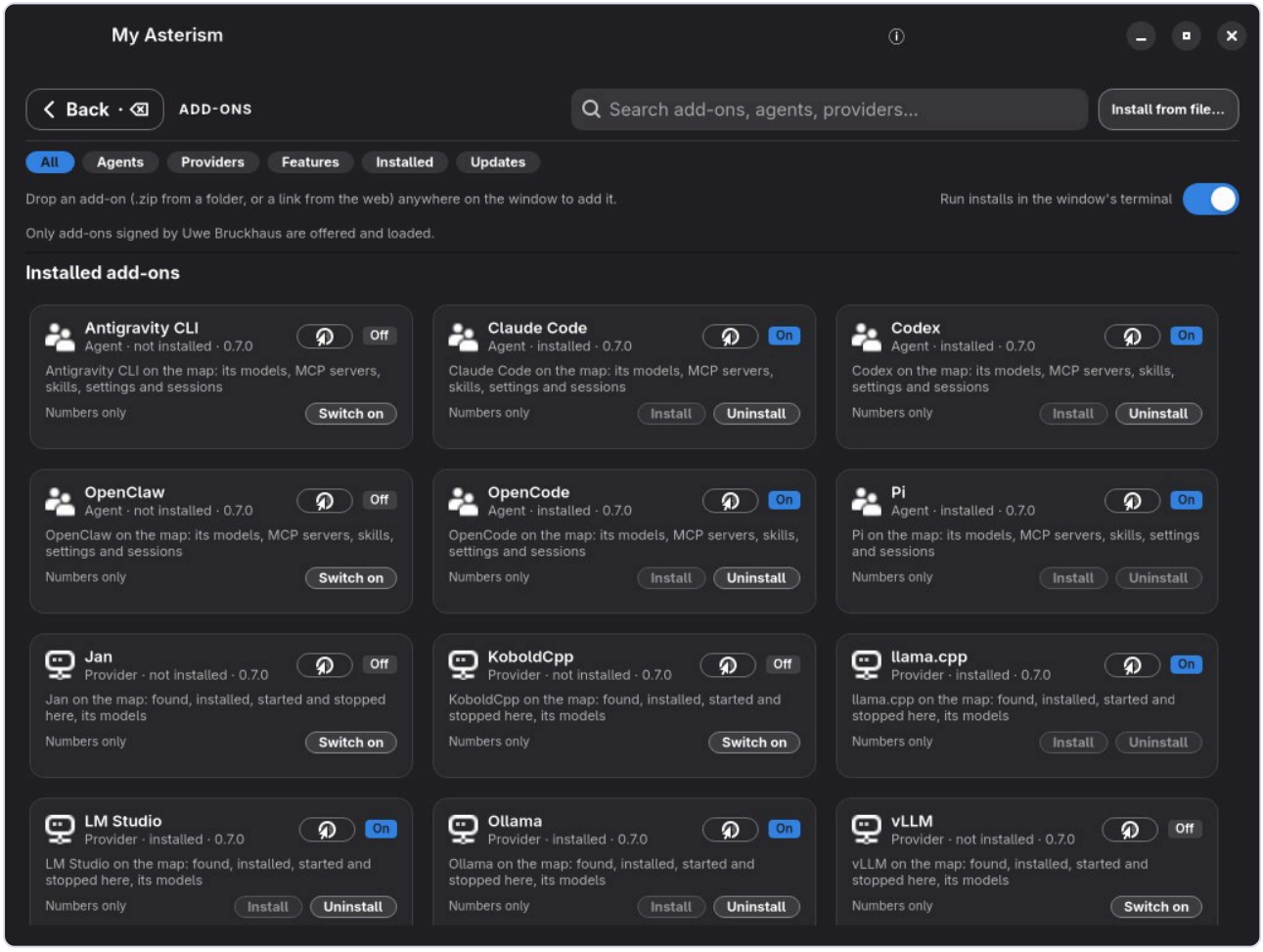


Figure 17 · Installed add-ons: on or off, with Install and Uninstall for the program.

The card of an agent or local LLM that is switched on has **Install** and **Uninstall** for the program itself (the one that does not apply is greyed out), and every card with a homepage has a globe button that opens it in your browser.

Search finds names, descriptions and what an add-on reads; the chips above narrow the list to agents, providers, features, the installed ones or those with an update. Click a card for its details: what it reads, what it sends, what it may do, and its privacy level.

17.3 Installing, switching on and off, updating, removing

Install (or **Get add-on**) first shows what the add-on reads, sends and may do; only after your OK is it downloaded, checked and switched on. **Switch off** and **Switch on** work at once, without a restart: its agents, providers, sections and cards appear or disappear everywhere. **Update** replaces it with the newer version from the catalog, after the new one was checked; **Remove** moves it to the trash. **Install from file…** takes an add-on zip you downloaded yourself; the same checks apply.

When the catalog has newer versions of installed add-ons, **Setup**'s cards say *Add-on update*, the open problems page (the red **!** in the header) lists **Add-on updates ready** with **Update all** and **Add-ons**, and the Add-ons page shows a bar with **Update all**. The card's ⋯ menu (or a right-click on it) also has **Update add-on to** *version*, for that one add-on. Update all updates every one of them in turn, with the same checks, and says in one message which were updated. One install or update runs at a time; another one started meanwhile asks you to try again in a moment.

> **ONLY FROM THE AUTHOR.** Maprisma loads only add-ons signed by Uwe Bruckhaus (key fingerprint `SHA256:e5cC634Ke3Our5LAeSc88qs22NKZEHTncBW5Ig3amWQ`). Before anything of an add-on runs, the signature is checked with OpenSSH's `ssh-keygen`, every file is compared with its SHA-256, and no file may have been added. A download must also match the SHA-256 in the signed catalog.

17.4 Privacy levels

Level	What the add-on may read or send
Numbers only	Counts, times, tokens, lines changed, cost — what Maprisma shows without add-ons.
Session titles	The titles an agent gave its sessions itself (never the conversations).
Titles and a summary by Claude	Those titles and numbers, sent to your own Claude Code — only on a click, after a preview of exactly what is sent.

An add-on can never do more than its level allows. Adding an agent or a provider, or installing a program, are separate permissions that the card names.

17.5 Kinds of add-ons

Kind	What it brings
Agent	One coding agent: where it keeps its settings, sessions, MCP servers and skills, how it is installed and how it signs in. Maprisma builds everything else from that — the card on Setup, its box on the map, Usage and tokens, Projects, Health and Security.

Kind	What it brings
Provider	A local LLM program, or the online providers: how it is found and installed, and how agents reach it. Maprisma builds its card, its box on the map and its details page — including **Usage and tokens**: what your agents sent through it, taken from their own session numbers.
Extension	More facts for an agent or a provider another add-on brings, without replacing it — for example its plan and limits, or a card on its details page. It only fills what the agent's add-on leaves empty; the add-on's page says what it adds to and anything it adds that is not used.
Feature	Something across agents, such as Insights (the Topics card).

An add-on's section on an agent's or a local LLM's details page joins the card it is about: Usage and tokens, Health, Topics (agents only), or a card of its own with its name.

An agent add-on describes its agent as facts (a file named `agent.toml`); most bring no code at all, and the add-on's page says so: *Brings no code*, or, for the few agents that need some (their own session database, say), *Facts about the agent, and a little code for what facts cannot say*.

17.6 A new agent on your computer

When you install an agent Maprisma does not know yet, it looks in the catalog (Settings › **Look for add-ons online**, on by default: the list is read from GitHub every few hours, nothing is sent but the request). If an add-on knows that agent, **Updates and suggestions** (the header's **!**) says so (*"Kilo Code is on this PC but not on the map"*) and a card under **Suggested for this PC**; one click installs it. If no add-on exists yet, the agent's details say so, with a link to ask for one.

17.7 Making an add-on

A new local LLM or a new agent needs nothing but an add-on: Maprisma builds its card, its box and lines on the map, its details, the Models page and the Security page from what the add-on says about it. If you can program a little, start from the templates in the Maprisma repository (`templates/`, explained in `docs/addons.md`): one file for a local LLM, a file of facts for an agent. Send the add-on to `github.com/uBruckhaus/asterism-addons`; once it is reviewed and signed, it appears under **Available** for everyone.

CHAPTER 18

Privacy and safety

- **No administrator rights for the app.** It never runs as root and never sees a password. Installing an agent that comes as a system package (such as Hermes or OpenClaw) runs the system's package manager through `sudo` in a terminal you watch, after showing you the command.
- **No credentials read.** It reads the agents' settings, but never their credential files. Whether an agent is signed in is judged from whether such a file exists. Values of keys and headers are never shown.
- **No conversations read.** Usage figures are read as numbers and times only. The one exception is watching an instance *In the background* (Terminals…), on your click, for that instance only; nothing of it is stored.
- **Network only when asked** — searching models, MCP servers or skills, installing, testing a server — plus an update check every few hours.
- **Checked downloads.** Programs are installed only when the download matches the checksum its release publishes.
- **Previews and Undo.** Every change to another program's settings is previewed, backed up and can be undone. Settings that may hold keys are written readable only by you.
- **Bounded.** Every program runs without a shell, and every answer is read with a size and time limit.
- **AI features only when you choose them.** Smart Check's look-ups run on a click, the daily AI checks only when switched on. The agent gets a short summary (names, versions, models), never keys or settings values, handed over in a way other users of the computer cannot read; its answers are checked before they are shown and never run. Tasks run in a terminal you watch, and the agent asks before each step. You use them at your own risk.

The README in each repository lists in full what is read, started and written.

CHAPTER 19

Troubleshooting

Problem	What to do
The map is cramped or cut off.	The window is narrower than about 1300 pixels. Make the window wider, or zoom out (`Ctrl`+`-`).
A box I expect is not on the map.	The map shows only what is set up: check **Setup** (an add-on switched off, an MCP server no agent uses) and the **Show** toggles.
Start here is missing or does nothing.	Install VTE for GTK 4 (`vte4`), or use **Start in terminal**.
A line is dotted red.	Click it for hints, fix what it says, then **Test again**. See When a connection fails.
The local LLM does not start with the agent.	Check Settings › **Start local LLMs with their agents**, and that the agent is connected on the map. `systemctl --user status ai-agents-map-providers` shows the watcher.
The coloured light from a local LLM to its model does not show.	It shows only while a program that is not on the map is generating with that LLM, and fades about ten seconds after its last request. Check the map is visible (a wide window) and that the program is actually sending something.
The agent answers badly or cuts off with a local model.	The context may be too small. Open the model under **Models… › Installed** and use **Set context** with the best value for your card.
A download stopped.	It resumes by itself after a broken connection. After a restart, use **Continue** in the strip at the top.
I want to undo a change from earlier.	Backups are kept in `~/.local/state/ai-agents-map/backups/`.

APPENDIX

Reference

A.1 Keyboard shortcuts

`F1`	Help
`Ctrl`+`,`	Settings
`F5`	Refresh (the map and Setup)
`Ctrl`+`+` / `Ctrl`+`-` / `Ctrl`+`0`	Zoom the map in, out, back to 100 %
`Ctrl`+scroll	Zoom the map
`Shift`+click	Add to or remove from the selection
`Esc`	Clear the selection; go back from a page
`Backspace`	Go back from a page

A.2 Keyboard use in the Flutter preview

The shortcuts above work in the Flutter preview too: `F1`, `Ctrl`+`,`, `F5`, `Ctrl`+`+` / `Ctrl`+`-` / `Ctrl`+`0` for the map in view, and `Esc` or `Backspace` to go back from a page. In a text field `Backspace` deletes, a dialog closes with `Esc`, and a terminal keeps both keys for its agent.

`Tab` moves through everything on the screen, `Enter` or `Space` uses it. On the map, `Enter` on a box opens it, and its ⋮ button opens its actions. In the map editor every drag has a way by keyboard: a box's **Actions** › **Connect**, then `Tab` to the target box and `Enter` (`Esc` cancels); **Move connection**, **Move up** and **Move down** are in the same menu. The app is checked for labelled controls, targets of at least 24 × 24 pixels and text contrast of at least 4.5:1, in light and dark.

A.3 Files and folders

Path	Contents
`~/.config/maprisma/` MAPRISMA `~/.config/omarchy/ai-agents-map/` OMARCHY	Settings, the MCP server list, looked-up agents, failed connections.
`~/.local/state/ai-agents-map/backups/`	Backups of every file the app changed.
`~/.local/state/ai-agents-map/jobs.json`	Interrupted downloads and installs.
`~/.local/state/ai-agents-map/reorganize.json`	The journal of the last reorganize, for Undo.

Path	Contents
`~/.cache/ai-agents-map/`	Scan results, tool lists, update checks, the last Smart Check (`smart-check.json`) and the AI checks' MCP updates (`mcp-updates.json`).
`~/.local/state/ai-agents-map/ai-help/`	The Protocol: one file per AI help run (the last 200).
`~/.local/state/omarchy-plugin-ai/runs.json`	The AI look-ups and tasks of this app and other plugins, with their tokens: Smart Check's statistics.
`~/.config/systemd/user/ai-agents-map-providers.service`	The watcher that starts local LLMs with their agents.

A.4 Glossary

Approval An agent asking before it changes a file or runs a command (see The words explained).

Context How many tokens of text a model can keep in mind at once.

GGUF The file format for models used by llama.cpp, LM Studio, Ollama and KoboldCpp.

Prompt injection Hidden instructions in a page, file or tool answer that try to steer an agent.

Quantization Storing a model with fewer bits per weight (for example *Q4_K_M*, about 4.8 bits) so it is smaller, with a small loss of quality.

Mixture of experts A model where only part of the parameters work for each token (*35B-A3B*: 35 billion in all, 3 billion active), which makes it fast for its size.

Sandbox A fenced-off area an agent's commands run in, reaching only the project folder and what you allow.

Token A piece of text, roughly three quarters of a word, that models read and write.

User service A background program started by systemd for your user only, without administrator rights.